
Learning to Approximate Uniform Facility Location via Graph Neural Networks

Chendi Qian¹ Christopher Morris¹ Stefanie Jegelka^{2,3} Christian Sohler⁴

Abstract

Neural networks, particularly message-passing neural networks (MPNNs), are increasingly used as heuristics for hard combinatorial optimization problems. Yet many learning-based methods rely on supervision, reinforcement learning, or gradient estimators, causing high computational cost, unstable training, or limited guarantees. Classical approximation algorithms provide worst-case guarantees but are non-differentiable and cannot adapt to structure in natural input distributions. We study this tradeoff through Uniform Facility Location (UniFL), a problem with applications in clustering, summarization, logistics, and supply chains. We propose a fully differentiable MPNN that incorporates approximation-algorithmic principles without solver supervision or discrete relaxations. The model has provable approximation guarantees and empirically improves on standard approximation algorithms, narrowing the gap to integer linear programming.

1. Introduction

Recent years have seen remarkable progress in applying neural networks to a variety of hard *combinatorial optimization* (CO) problems. Message-passing graph neural networks (MPNNs) (Gilmer et al., 2017; Scarselli et al., 2009) have been particularly popular due to their permutation invariance, scalability, and ability to exploit graph sparsity, making them natural candidates for leveraging structural information across CO instances. MPNNs have been used both as end-to-end heuristic solvers, e.g., Bello et al. (2016); Karalias & Loukas (2020); Khalil et al. (2017), and as learned components inside classical exact solvers, such as branch-and-cut pipelines for integer-linear optimization or SAT solving. In the latter setting, an MPNN replaces node-

, variable-, or cut-selection heuristics (Gasse et al., 2019; Khalil et al., 2022; Scavuzzo et al., 2024; Tönshoff & Grohe, 2025); see Cappart et al. (2023) for a survey.

The motivation for combining learning and algorithms is to discover better heuristics and adapt to instance distributions, yielding faster, higher-quality solutions in practice by exploiting recurring structure in the data. However, two challenges repeatedly arise: (i) how to train these models effectively and efficiently, and (ii) how to ensure reliable solution quality.

Regarding training, two core difficulties are common. First, supervision with optimal solutions is expensive because computing such solutions is itself hard. Second, CO objectives are typically discrete and therefore non-differentiable. To handle non-differentiability, several works adopt surrogate gradient estimators (Petersen et al., 2024), including straight-through estimators (Bengio et al., 2013), continuous relaxations based on the Gumbel–softmax trick or concrete distribution (Jang et al., 2017; Maddison et al., 2017), and more advanced gradient-estimation techniques such as I-MLE (Niepert et al., 2021) or SIMPLE (Ahmed et al., 2023). In practice, however, these methods can be brittle or cumbersome to tune. Another line of work (Karalias & Loukas, 2020; Karalias et al., 2022; 2025) trains directly on a continuous extension of the discrete objective, circumventing both supervision and differentiability barriers.

Yet, these approaches, and end-to-end learned solvers more broadly, typically do not provide guarantees on the quality of the returned solutions. This stands in sharp contrast to *approximation algorithms*, which provide polynomial-time algorithms with rigorous worst-case guarantees on solution quality (Vazirani, 2001). Such guarantees imply strong out-of-distribution robustness, since they hold for all inputs. The downside is that classical approximation algorithms are usually distribution-agnostic, i.e., they are designed for worst-case instances and therefore do not directly exploit structural regularities present in realistic data distributions, where substantially better performance may be possible.

Recently, *algorithms with predictions* (Mitzenmacher & Vasilvitskii, 2020) have aimed to incorporate learning-based predictions into discrete algorithms. However, these methods largely treat predictions as black boxes and are typically non-differentiable, making them difficult to integrate into

¹RWTH Aachen University, Germany ²Technical University of Munich, Germany ³Massachusetts Institute of Technology, USA ⁴University of Cologne, Germany. Correspondence to: Chendi Qian <chendi.qian@log.rwth-aachen.de>.

fully differentiable neural pipelines. Another line of work integrates GNNs into classical exact solvers, e.g., (Gasse et al., 2019), thereby providing certificates for the final solutions. However, training such systems can be extremely resource-intensive, since it often requires repeatedly running a full solver—hundreds or thousands of times—to generate supervision signals or to evaluate learned policies. This dependence on repeated calls to expensive optimization routines limits scalability to large graphs and large training sets.

This gap between robust but conservative approximation algorithms and expressive but potentially unstable (or costly) learned solvers motivates us to combine the strengths of both paradigms. Ideally, one seeks architectures that are (i) fully differentiable and easy to train, (ii) unsupervised or weakly supervised to avoid costly solver calls, (iii) equipped with provable approximation guarantees, and (iv) able to exploit distribution-specific structure to improve empirical performance beyond the training set. Achieving these goals simultaneously remains a major open challenge, with very little existing work (Yau et al., 2024).

Contributions In this work, we take a step toward this vision for the exemplary, fundamental case of the *uniform facility location problem* (UniFL). The UniFL asks to open a subset of facilities in a given finite metric space, each with uniform opening cost, to serve a set of clients, minimizing the sum of opening and connection costs. It has wide-ranging applications in logistics, supply-chain design, clustering, and network design (Lammersen, 2010). In machine learning, facility location-like objectives have also been used for data summarization (Wei et al., 2014; Bairi et al., 2015). Even in this simplified case, with uniform opening costs, facility location is hard to approximate with a polynomial-time approximation algorithm (Bădoiu et al., 2005). Hence, we propose an MPNN-based framework that integrates approximation-algorithmic insights with differentiable learning, exploiting distributional information, while still providing theoretical performance guarantees; see Figure 1 for a schematic overview of our framework. Concretely,

1. we introduce a novel fully differentiable, unsupervised MPNN architecture tailored to UniFL—designed to mirror key structural components of classical approximation algorithms. In addition, a slight change in the cost function enables end-to-end k -means clustering.
2. We show how to initialize the model with parameters that yield *constant-factor approximation guarantees* and can be improved during training—bridging the gap between approximation algorithms and neural network-based heuristics.
3. We show that we can train the architecture on a finite

training set and that it provably generalizes to UniFL instances of the same size.

4. Empirically, we show that architecture outperforms classical, non-learned approximation algorithms and is competitive to state-of-the-art branch-and-cut solvers while generalizing reliably to problem instances significantly larger than those seen during training.

In summary, our approach embeds approximation-algorithmic structure into a fully differentiable, unsupervised MPNN, yielding a provably approximate method that generalizes well and outperforms classical baselines.

Conflict of Interest Disclosure We declare that we have no specific financial conflicts of interest to disclose related to this work.

1.1. Related work

In the following, we discuss related work.

MPNNs MPNNs (Gilmer et al., 2017; Scarselli et al., 2009) emerged as the most prominent graph machine learning architecture. Notable instances of this architecture include, e.g., Duvenaud et al. (2015); Hamilton et al. (2017); Kipf & Welling (2017), and Veličković et al. (2018), which can be subsumed under the message-passing framework introduced in Gilmer et al. (2017). In parallel, approaches based on spectral information were introduced in, e.g., Bruna et al. (2014); Defferrard et al. (2016); Gama et al. (2019); Kipf & Welling (2017); Levie et al. (2019), and Monti et al. (2017)—all of which descend from early work in Baskin et al. (1997); Goller & Küchler (1996); Kireev (1995); Merkwirth & Lengauer (2005); Micheli & Sestito (2005); Micheli (2009); Scarselli et al. (2009), and Sperduti & Starita (1997).

Facility location problem The metric uncapacitated facility location problem is a classical topic in approximation algorithms (Shmoys et al., 1997; Shmoys, 2000). A long line of work developed constant-factor approximation algorithms (see (Shmoys et al., 1997; Charikar & Guha, 1999; Mahdian et al., 2006; Arya et al., 2004; Gupta & Tangwongsan, 2008; Mettu & Plaxton, 2003; Thorup, 2004; Jain & Vazirani, 2001) for a few representative results) using linear programming-based rounding, primal-dual methods, and local search; see Shmoys (2000) for an overview. The problem is APX-hard, and no polynomial-time algorithm can achieve a factor better than 1.463 unless $P = NP$ (Guha & Khuller, 1999). In constant-dimensional Euclidean space, there is a polynomial-time approximation scheme that was later extended to metric spaces with bounded doubling dimension. Also, distributed approximation algorithms are known (Moscibroda & Wattenhofer, 2005), including an $\mathcal{O}(1)$ -approximation algorithm in the CONGEST model on a clique (Berns et al., 2012). Even the UniFL, where all opening costs are identical, remains NP-hard, yet its simplified

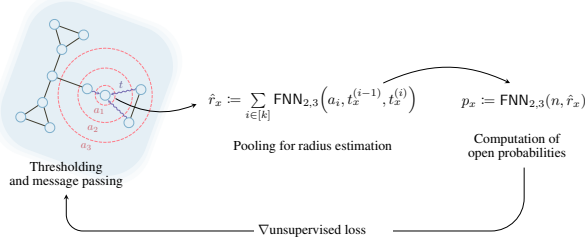


Figure 1. Overview of how MPNN probably recovers a near-optimal solution for the UniFL problem. Local message-passing around a given point x is used to compute an estimate of the radius r_x , followed by the computation of the opening probabilities, and MPNN’s parameters are computed based on an unsupervised loss via the expected cost of the solution.

structure admits efficient local and distributed algorithms. In particular, the radius-based characterization introduced by Mettu & Plaxton (2003), refined in Bădoiu et al. (2005), and the distributed algorithm $\mathcal{O}(1)$ -UFL (Gehweiler et al., 2014) yield constant-factor approximation guarantees using only local neighborhood information. Our work builds on these ideas by embedding such local decision rules into a differentiable MPNN architecture, enabling provable approximation guarantees while allowing data-driven refinement of classical algorithmic techniques.

See Appendix B for an extended discussion on related work.

2. Background

In the following sections, we introduce the notation and provide the necessary background.

Basic notations Let $\mathbb{N} := \{1, 2, \dots\}$. The set $\mathbb{R}^+$ denotes the set of non-negative real numbers. For $n \in \mathbb{N}$, let $[n] := \{1, \dots, n\} \subset \mathbb{N}$. We use $\{\dots\}$ to denote multisets, i.e., the generalization of sets allowing for multiple, finitely many instances of each of its elements.

Graphs An (undirected) graph G is a pair $(V(G), E(G))$ with finite sets of vertices $V(G)$ and edges $E(G) \subseteq \{\{u, v\} \subseteq V(G) \mid u \neq v\}$. The order of a graph G is its number $|V(G)|$ of vertices. If not stated otherwise, we set $n := |V(G)|$ and call G an n -order graph. Let $\mathcal{G}$ be a set of graphs, then $\mathcal{V}(\mathcal{G})$ denotes the set of pairs (G, v) where $G \in \mathcal{G}$ and $v \in V(G)$. The neighborhood of a vertex $v \in V(G)$ is denoted by $N_G(v) := \{u \in V(G) \mid \{v, u\} \in E(G)\}$, where we usually omit the subscript for ease of notation, and the degree of a vertex v is $|N_G(v)|$. An attributed graph is a pair (G, a_G) with a graph G and an (vertex-)attribute function $a_G: V(G) \rightarrow \mathbb{R}^d$, for $d > 0$. The attribute or feature of $v \in V(G)$ is $a_G(v)$. An edge-weighted (attributed) graph is a pair (G, w_G) with a (attributed) graph G and an (edge-)weight function $w_G: E(G) \rightarrow \mathbb{R}^+$. For an edge $e \in E(G)$, $w_G(e)$ is the (edge) weight of e .

Here, we define metric spaces, which play an essential role in the following.

Metric spaces Let $\mathcal{X}$ be a set equipped with a pseudo-metric $d: \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$, i.e., d is a function satisfying $d(x, x) = 0$ and $d(x, y) = d(y, x)$ for $x, y \in \mathcal{X}$, and $d(x, y) \leq d(x, z) + d(z, y)$, for $x, y, z \in \mathcal{X}$. The latter property is called the triangle inequality. The pair $(\mathcal{X}, d)$ is called a pseudo-metric space. For $(\mathcal{X}, d)$ to be a metric space, d additionally needs to satisfy $d(x, y) = 0 \Rightarrow x = y$, for $x, y \in \mathcal{X}$. For $x \in \mathcal{X}$, and $Y \subseteq \mathcal{X}$ let $B_Y(x, r) := \{y \in Y: d(x, y) \leq r\}$. If no ambiguity can arise, we also write $B(x, r)$ instead of $B_{\mathcal{X}}(x, r)$.

See Appendix A for additional details on continuity assumptions, MPNNs, and feed-forward neural networks.

2.1. Uniform facility location

Here, we formally introduce the uniform facility location problem (UniFL). To that, let $(\mathcal{X}, d)$ be a finite metric space, with $|\mathcal{X}| = n$. In the UniFL, the objective is to find a set $F \subseteq \mathcal{X}$ such that

$$\sum_{x \in \mathcal{X}} d(x, F) + |F| \quad (1)$$

is minimized, where $d(x, F) := \min_{y \in F} d(x, y)$. That is, our objective is to find a set F of facilities to open, where facilities can be opened at any location $x \in \mathcal{X}$, and, w.l.o.g., each facility costs a uniform 1. The points $x \in \mathcal{X}$ at which we did not open a facility are the clients, and they are served by the closest facility f_x at cost $d(x, f_x)$. For a given metric space S , we denote by OPT_S the smallest possible value of Equation (1). We remark that we can replace the unit opening cost by an arbitrary value $\lambda > 0$ and minimize $\sum_{x \in \mathcal{X}} d(x, F) + \lambda \cdot |F|$ as we can simply rescale all distances by a factor $1/\lambda$ and obtain the version with opening costs 1. With this formulation, we may interpret the problem as a clustering problem with a regularization term to control the number of clusters.

In the following, we outline some known results that are useful for the UniFL. To that, let $(\mathcal{X}, d)$ be a finite metric space and $x \in \mathcal{X}$. Following Mettu & Plaxton (2003), the radius of x is the value $r_x > 0$ such that

$$\sum_{y \in \mathcal{X} \cap B(x, r_x)} r_x - d(y, x) = 1. \quad (2)$$

See Figure 2 in the appendix for an illustration of the radius. Interestingly, knowing the radii of all points in a given metric space S is useful for determining the minimum value OPT_S , as stated in the following result.

Lemma 1 ((Bădoiu et al., 2005)). *Let $S := (\mathcal{X}, d)$ be a finite metric space, then $\sum_{x \in \mathcal{X} \cap B(x, r_x)} r_x \in \Theta(\text{OPT}_S)$.*

Hence, knowing the radius leads to a constant-factor approximation of the smallest possible value of OPT_S .

Graph representation of UniFL instances We can straightforwardly encode a given UniFL instance $S := (\mathcal{X}, d)$ as an edge-weighted graph G_S where $V(G_S) := \mathcal{X}$ and $E(G_S) := \{(u, v) : d(u, v) \leq 1\}$ contain edges for every pair of vertices u and v that are at a distance smaller than 1. Note that we remove edges of weight greater than 1 because, following Equation (1), it is cheaper to open a facility than to serve it from a facility at a distance of more than 1. The edge weights $w : E(G_S) \rightarrow \mathbb{R}$ of G_S are the distances between the vertices, i.e., $w((u, v)) := d(u, v)$, for $(u, v) \in E(G_S)$. Let $\mathcal{G}_n$ denote the set of graphs encoding UniFL instances of finite metric spaces with $|\mathcal{X}| = n$. If the edge-weights are from a compact subset of $\mathbb{R}$ we write $\mathcal{G}_n^C$.

3. An expressive and generalizing MPNN architecture for approximating the UniFL

Here, we devise a simple, scalable MPNN architecture that can represent an $\mathcal{O}(\log(n))$ -approximation algorithm for the UniFL with input size n while being trainable in an unsupervised setting. Subsequently, we extend the algorithm to a constant-factor approximation algorithm for the UniFL. We start by devising a distributed approximation algorithm for UniFL and neuralize it, while retaining the ability to recover the approximation factor for the non-learned algorithm. Finally, we show that a finite training dataset is sufficient to learn parameters that simulate the algorithm on the UniFL instance of arbitrary input size.

3.1. A simple distributed algorithm for the UniFL

In the following, we derive a distributed $\mathcal{O}(\log(n))$ -approximation algorithm for the UniFL, which essentially leverages Lemma 1. Subsequently, we extend to a constant-factor approximation algorithm. That is, given a finite metric space $(\mathcal{X}, d)$, we first compute the radius for each $x \in \mathcal{X}$ and then open a facility with probability $\min(1, c \cdot \ln n \cdot r_x)$, for an appropriate constant $c > 0$. Afterward, points $x \in \mathcal{X}$ that do not have a facility opened at a distance strictly smaller than 1 open up a facility themselves; see SIMPLEUNIFORMFL for the pseudo-code of the algorithm.

SIMPLEUNIFORMFL($\mathcal{X}, d$)

1. **for each** $x \in \mathcal{X}$ **do in parallel** compute the radius r_x
2. **for each** $x \in \mathcal{X}$ **do in parallel**
independently open a facility with probability $\min(1, c \cdot \ln n \cdot r_x)$ for an appropriate constant $c > 0$
3. Let F_1 be the set of open facilities
4. **for each** $x \in \mathcal{X}$ **do in parallel**
5. **if** there is an open facility f in F_1 with $d(x, f) < 1$
 then assign x to f
6. **else** open a facility at x

7. Let F_2 be the set of facilities opened in line 6
8. **return** $F_1 \cup F_2$

The following results establish the $\mathcal{O}(\log(n))$ -approximation result of the above algorithm SIMPLEUNIFORMFL.

Proposition 2. *Let $S = (\mathcal{X}, d)$ be a UniFL instance, then the algorithm SIMPLEUNIFORMFL with $c \geq 2$ computes a solution to the UniFL for S with expected cost $\mathcal{O}(\log(n)) \cdot \text{OPT}_S$, where OPT_S denotes the cost of an optimal solution.*

We remark at this point that the distributed algorithm $\mathcal{O}(1)$ -UFL in Gehweiler et al. (2014) achieves a constant approximation in expectation. However, our MPNN implementation computes the opening probabilities, and the algorithm’s simplicity allows us to estimate the expected cost of the corresponding solution. This will be crucial for our training procedure. We are not aware of a way to do this for the algorithm from (Gehweiler et al., 2014). In Section 3.3, we argue that we can modify the algorithm in such a way that it only assigns a subset of $\mathcal{X}$ to open facilities and can be recursively applied to the remaining instances in such a way that the resulting (combined over the recursive calls) solution is a constant approximation in expectation.

3.2. A simple MPNN architecture for UniFL

In the following, we derive a simple MPNN architecture that is expressive enough to simulate SIMPLEUNIFORMFL; see Figure 1 for a schematic overview. For each point x in the given metric space, following Equation (6) in the appendix, the MPNN aggregates information from the local neighbors to compute a lower bound for the radius r_x , which is subsequently used to compute opening probabilities for each point x . Unlike SIMPLEUNIFORMFL, every step of the architecture is parameterized and fully differentiable, allowing for unsupervised end-to-end training on a given instance distribution.

Formally, let $S := (\mathcal{X}, d)$ denote a finite metric space, $x \in \mathcal{X}$ be a client, and, w.l.o.g., we assume $r_x \leq 1$. To compute the radius of x , we discretize the range $(0, 1]$ into “bins” and determine into which bin r_x falls. To that, for $k > 0$, let $(a_0, a_1, \dots, a_k)$ be a tuple of real numbers, a *discretization*, such that $0 = a_0 < a_1 < a_2 < \dots < a_k = 1$. We can now estimate the radius via the following local aggregation,

$$\begin{aligned} t_x^{(i)} &:= \min \left\{ 1, \sum_{y \in N(x)} \text{reLU}(a_i - d(x, y)) \right\} \\ &= 1 - \text{reLU} \left(1 - \sum_{y \in N(x)} \text{reLU}(a_i - d(x, y)) \right), \end{aligned}$$

for $i \in [k]$, which we can further parametrize with a two-

layer FNN using ReLU activation functions as follows,

$$t_x^{(i)} \equiv \text{FNN}_{2,3} \left(\sum_{y \in N(x)} \text{FNN}_{1,3}(a_i, d(x, y)) \right), \quad (3)$$

where the existence of the required weight assignments is easily verified. The idea is that $t_x^{(i)}$ is equal to 1 if the radius is at least a_i . Hence, we compute an estimate of the radius

$$\begin{aligned} \hat{r}_x &:= \sum_{i \in [k]} a_i \cdot (t_x^{(i-1)} - t_x^{(i)}) \\ &\equiv \sum_{i \in [k]} \text{FNN}_{2,3} \left(a_i, t_x^{(i-1)}, t_x^{(i)} \right), \end{aligned} \quad (4)$$

via parameterized local aggregation, and estimate the probability of making client x a facility via

$$p_x := \min\{1, c \cdot \log(n) \cdot \hat{r}_x\} \equiv \text{FNN}_{2,3}(n, \hat{r}_x).$$

Training to solve UniFL In the following, we devise an unsupervised loss that allows training to adapt the parameters of the above MPNN architectures to UniFL instance distributions. To that, let F_1 denote the random variable for the set of facilities opened by the algorithm in line 2, and F_2 be the random variable for the set of facilities opened by the algorithm in line 7 in the algorithm SIMPLEUNIFORMFL. We first observe that a facility is opened if it belongs to F_1 or F_2 . Let A_f denote the indicator random variable for the event that $f \in F_1$ and let B_f denote the indicator random variable for the event $f \in F_2$, i.e., there is no facility in F_1 that has a distance smaller than 1. The expected opening cost can then be written as

$$\mathbb{E} \left[\sum_{f \in X} A_f + \sum_{f \in X} B_f \right] = \sum_{f \in X} \mathbb{E}[A_f] + \sum_{f \in X} \mathbb{E}[B_f]$$

by linearity of expectation. We further have $\mathbb{E}[A_f] = \Pr[A_f = 1] = p_f$ and $\mathbb{E}[B_f] = \Pr[B_f = 1] = \prod_{x \in X; d(x, f) < 1} (1 - p_x)$. Thus, the expected opening cost is

$$\sum_{f \in X} p_f + \sum_{f \in X} \prod_{x \in X; d(x, f) < 1} (1 - p_x).$$

Recall that we only connect clients to F_1 . To obtain the expected connection cost at a point $x \in X$, we observe that if there is no $f \in F_1$ within distance 1 of x , the facility opens and incurs no connection cost. If this is not the case, we connect to the nearest open facility from F . A facility f is the nearest open facility to a point x , if f is open and all facilities that are closer to f are closed. The latter happens with probability $p_f \cdot \prod_{z \in X; d(x, z) < d(x, f)} (1 - p_z)$. Hence,

overall, we get the following expected costs

$$\begin{aligned} \mathbb{E}[\text{cost}(X, F)] &= \sum_{f \in X} p_f + \sum_{f \in X} \prod_{x \in X; d(x, f) < 1} (1 - p_x) \\ &+ \sum_{x \in X} \sum_{f \in X; d(x, f) < 1} \text{dist}(x, f) \cdot p_f \\ &\cdot \prod_{z \in X; d(x, z) < d(x, f)} (1 - p_z), \end{aligned} \quad (5)$$

where we assume that all pairwise distances are distinct. For a finite metric space $(\mathcal{X}, d)$ with $n = |\mathcal{X}|$ and the corresponding graph representation G_S having at most degree d , the running time for evaluating the above loss is in $\mathcal{O}(nd^2)$.

Approximation guarantees In the following, we show that there are parameters for the above architecture such that we can recover the $\mathcal{O}(\log(n))$ -factor approximation of the Proposition 2 for the non-learned algorithm SIMPLEUNIFORMFL. Precisely, we show the following result.

Proposition 3. *For every $n \in \mathbb{N}$, there exists a $k \in \mathbb{N}$, a discretization $0 = a_0 < a_1 < \dots < a_k = 1$, and an MPNN architecture $\text{mpnn}_\theta: \mathcal{V}(\mathcal{G}_n) \rightarrow [0, 1]$ parameterized by θ , following Section 3.2, and a parameter assignment θ^* such that for every UniFL instance $S := (\mathcal{X}, d)$ with $|\mathcal{X}| = n$, the MPNN mpnn_{θ^*} outputs vertex-wise probabilities*

$$p_v := \text{mpnn}_{\theta^*}(G_S, v) \in [0, 1], \quad \text{for } v \in V(G_S),$$

such that

$$\mathbb{E}[\text{cost}(\mathcal{X}, p)] \leq \mathcal{O}(\log(n)) \cdot \text{Opt}_S,$$

where $\text{cost}(X, p)$ denotes the facility-opening and connection cost obtained by applying the standard SimpleUniFL post-processing (line 4–6 in SIMPLEUNIFORMFL) using the probabilities $\{p_x\}_{x \in X}$. In particular, the mpnn_{θ^*} realizes an $\mathcal{O}(\log(n))$ -approximation algorithm for the UniFL on all instances of order n .

For our setting where the algorithm is constrained to compute opening probabilities for the facilities and the solution is obtained by opening the facilities independently with these probabilities, we show that there cannot exist a constant-depth (deterministic) MPNN architecture that achieves an approximation factor of $\mathcal{O}(\log(n))$.

Proposition 4. *Let mpnn be an arbitrary (deterministic) MPNN that*

1. *receives as input an edge-weighted graph G_S that encodes an instance $S := (\mathcal{X}, d)$ of the uniform facility location problem, and*
2. *computes opening probabilities p_v for every vertex $v \in V(G_S)$.*

Then there is n_0 such that for every $n \geq n_0$ there exists an instance finite metric space $S := (\mathcal{X}, d)$, with $|\mathcal{X}| = n$, such that

$$\mathbb{E} \left[|F| + \sum_{p \in \mathcal{X}} \min\{1, \text{dist}(p, F)\} \right] \geq \frac{\ln(n)}{2} \cdot \text{Opt}_S.$$

3.3. Recursively solving UniFL with GNNs

Above, we showed that there exists a constant depth and width MPNN architecture that leads to a $\mathcal{O}(\log(n))$ -factor approximation for the UniFL. In the following, we extend this to constant-factor approximation. To that, we first devise a discrete constant-factor approximation that recursively applies SIMPLEUNIFORMFL. To that, we will slightly modify our approach from the previous section, allowing us to leave some facilities unassigned. The sets of open and unassigned facilities are returned. The algorithm may therefore also receive a set of facilities as input, expected to come from an earlier invocation; see UNIFORMFLRECURSIONSTART for the algorithm's pseudo-code.

UNIFORMFLRECURSIONSTART($\mathcal{X}, d$)

1. $F = \emptyset; X = \mathcal{X}$
2. **while** $X \neq \emptyset$ **do**
3. $S, R = \text{RECURSIVEUNIFORMFL}(X, F, d)$
4. $F = F \cup S$
5. $X = R$
6. **return** F

RECURSIVEUNIFORMFL($\mathcal{X}, F, d$)

1. **for each** $x \in \mathcal{X}$ **do in parallel** compute the radius r_x
2. **for each** $x \in \mathcal{X}$ **do in parallel**
3. independently open a facility at x with probability $\min\{1, c \cdot d(x, F), c \cdot r_x\}$
for an appr. constant $c > 0$
4. Let F be the set of open facilities
5. $R = \emptyset$
6. **for each** $x \in \mathcal{X}$ **do in parallel**
7. **if** there is a facility f in F with $d(x, f) \leq 6r_x$
 then assign x to the closest $f \in F$
8. **else** $R = R \cup \{x\}$
9. **return** F, R

3.4. Learning to solve UniFL from finite data

While Section 3.2 shows that there exist parameters such that an MPNN can achieve an $\mathcal{O}(\log(n))$ -factor approximation to the UniFL, it is unclear how to learn a generalizing parameter assignment that works for all metric spaces of a given size. Hence, the following result show that for every size $n \in \mathbb{N}$, there exists an MPNN, a finite training set, and a regularization term such that minimizing the loss and the

regularization term leads to a parameter assignment that recovers the opening probabilities of the MPNN architecture devised in Proposition 3, leading to an $\mathcal{O}(\log(n))$ -factor approximation algorithm for any finite metric space of size n .

Proposition 5. For every $n \in \mathbb{N}$, there exists an MPNN architecture $\text{mpnn}_{\theta}^{(n)}: \mathcal{V}(\mathcal{G}_n^C) \rightarrow [0, 1]$, with parameters θ , such that for $\varepsilon \in (0, 1)$, there is $\varepsilon' > 0$, and a finite training datasets $T_{n, \varepsilon} \subset \mathcal{V}(\mathcal{G}_n^C) \times \mathbb{R}$ consisting of pairs $((G, v), p_v)$, where $G \in \mathcal{G}_n^C$ is a finite, edge-weighted graph encoding an UFL instance, $v \in V(G)$, and p_v is the corresponding opening probability according to Proposition 3, and a differentiable regularization term r_n such that, such that

$$\frac{1}{|T_{n, \varepsilon}|} \sum_{((G, v), p_v) \in T_n} |\text{mpnn}_{\theta}^{(n)}(G, v) - p_v| + r_n(\theta) \leq \varepsilon'$$

implies

$$\|p_{G_S} - h_{\theta}(G_S)\| \leq \varepsilon,$$

for all $G_S \in \mathcal{G}_n^C$ encoding UniFL instances.

4. Experimental study

Here, we empirically assess the extent to which our theoretical results translate into empirical results and whether we can achieve improved empirical approximation ratios relative to the theoretical worst-case bounds. Concretely, we answer the following questions.

Q1 Does the MPNN architecture from Section 3.2 lead to improved approximation compared to the non-learned SIMPLEUNIFORMFL?

Q2 Do the size generalization results of Proposition 5 translate into small generalization error on larger instances than seen during training?

Q3 Can the MPNN architecture also compete with scalable clustering methods such as KMeans++ and KMedoids++?

Data generation We generate synthetic random geometric graphs to evaluate the model's performance with 2, 5, or 10 dimensions. Each graph consists of 1000 vertices. Node positions are sampled from a Gaussian mixture model with 100 isotropic Gaussian components and random centroids. We construct the graph structure by creating an edge (u, v) within a unit Euclidean distance. Additionally, self-loops are added to all vertices to capture situations in which a facility serves itself. We generate 10 000 such graphs for each dataset, splitting them into training, validation, and test sets with a 8:1:1 ratio. Furthermore, to evaluate size generalization, we generate additional test sets with graph sizes ranging from 2000 to 10 000 vertices, with the same Euclidean dimensions and an average node degree approximately equal to that of the training set. Detailed statistics,

including node counts and average degrees, are provided in Table 8.

To assess applicability to realistic structures, we utilize the city map dataset introduced by Liang et al. (2025). This collection consists of four large-scale graphs representing metropolitan road networks, where vertices correspond to junctions and edges represent roads weighted by their lengths. We preprocess these graphs by normalizing the edge weights so that the maximum road length in the network is 1. Notably, these graphs are not strictly geometric, as edge weights may violate the triangle inequality due to the nonlinear geometry of roads (e.g., detours). However, the dataset remains valuable for real-world applicability, where optimal facility location is constrained by realistic edge weights rather than ideal Euclidean distance.

Experimental protocol We formulate the UniFL as an integer-linear program (see Appendix D for details), to compute the optimal solution. Besides, we leverage the algorithms SIMPLEUNIFORMFL and UNIFORMFLRECURSIONSTART as baselines. They are fully stochastic and serve as a non-learnable counterpart of our MPNN architectures. The constant hyperparameter c of them is tuned via grid search. Furthermore, we include Algorithm 3.1 ($\mathcal{O}(1)$ -UFL) in Gehweiler et al. (2014) as a tuning-free baseline. The baselines are all evaluated by averaging 1000 samples with fixed seeds per instance.

For the neural method, we train an MPNN using the proposed unsupervised loss in Equation (5) with 5 different seeds. During inference, the trained model guides sampling in SIMPLEUNIFORMFL, and we also report the expected cost over 1000 samples. To evaluate performance on the k -Means objective, we introduce a variant based on Equation (5), replacing the distance in the last term with squared Euclidean distance. We compare this against standard k -Means and k -Medoids baselines, fixing the number of clusters k to the MPNN’s predicted facility count to ensure a fair comparison.

We report the cost components and the optimality ratio relative to the ILP solver. Computational efficiency is measured as the wall-clock time for a single solution generation. That is, for baselines, we compute the radii, while for the MPNN-guided approach, we measure the MPNN inference time, plus one sampling step. Experiments were conducted on an NVIDIA L40S GPU (for neural and stochastic methods) and an Intel® Xeon® Silver 4510 CPU (for ILP and k -Means). A detailed description of hyperparameters, architecture, and experiment protocol is provided in Appendix D.

4.1. Results and discussion

Here, we present results that answer questions **Q1** to **Q3**.

Approximations ratios and computation times (Q1) To

Table 1. Performance comparison on synthetic geometric graphs in various dimensions. Results of stochastic methods are averaged over 1000 samples. Besides, MPNN results show the mean $\pm$ standard deviation over five training runs.

Candidate	Cost	Geo-1000-2	Geo-1000-5	Geo-1000-10	Geo-1000-10-sparse	Geo-1000-10-dense
Solver	Open	366.302	369.884	291.488	686.578	158.483
	Con.	279.827	403.072	520.611	265.496	509.294
	Total	646.129	772.956	812.099	952.074	667.777
	Time	0.263	0.299	0.689	0.089	8.179
SimpleUFL	Open	512.082	523.173	436.602	816.256	243.329
	Con.	241.435	320.425	431.908	155.263	499.281
	Total	753.517	843.598	868.511	971.519	742.610
	Time	0.001	0.001	0.001	0.001	0.001
RecursiveUFL	Open	350.491	414.067	367.162	734.623	221.263
	Con.	367.832	415.522	496.749	227.582	520.609
	Total	718.323	829.589	863.912	962.205	741.872
	Time	0.015	0.006	0.004	0.014	0.003
$\mathcal{O}(1)$ -UFL	Open	613.904	715.915	731.670	878.267	653.293
	Con.	148.977	168.827	191.341	102.478	190.738
	Total	762.881	884.742	923.011	980.746	844.032
	Time	0.001	0.001	0.001	0.001	0.001
MPNN	Open	380.816 $\pm$ 0.361	381.572 $\pm$ 0.070	298.689 $\pm$ 1.134	711.482 $\pm$ 0.082	159.510 $\pm$ 2.513
	Con.	271.317 $\pm$ 0.738	394.342 $\pm$ 0.072	515.191 $\pm$ 1.301	243.491 $\pm$ 0.184	510.431 $\pm$ 2.369
	Total	652.133 $\pm$ 0.025	775.914 $\pm$ 0.015	813.879 $\pm$ 0.048	954.973 $\pm$ 0.003	669.942 $\pm$ 0.056
	Time	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.006 $\pm$ 0.000
Ratio	Open	1.009 $\pm$ 0.000	1.003 $\pm$ 0.000	1.002 $\pm$ 0.000	1.003 $\pm$ 0.000	1.003 $\pm$ 0.000
	Con.					
	Total					
	Time					

answer **Q1**, the quantitative results on synthetic geometric graphs are presented in Table 1, detailing the facility opening cost, connection cost, total cost, optimality ratio, and execution time. As expected, the stochastic heuristics SIMPLEUNIFORMFL and $\mathcal{O}(1)$ -UFL (Gehweiler et al., 2014) exhibit higher costs. A closer look at the cost reveals that $\mathcal{O}(1)$ -UFL (Gehweiler et al., 2014) tends to open more facilities than necessary, thereby drastically reducing connection costs but increasing total costs. The recursive refinement in UNIFORMFLRECURSIONSTART mitigates this, achieving ratios mostly within the 10% theoretical bound in Lemma 12. Slight deviations are observed in both the 2D and dense 10D datasets. Most notably, our MPNN architecture consistently outperforms all baselines, achieving near-optimal ratios across all datasets. Crucially, the MPNN also reproduces the balance between opening and connection costs. For instance, on Geo-1000-5, the exact solver gives opening/connection costs at roughly a 370/403 ratio, while our MPNN architectures arrive at a similarly 381/394, indicating the usefulness of the unsupervised loss in Equation (5). Furthermore, the MPNN architecture achieves high precision with negligible computational overhead, with the inference time of 4ms on a GPU.

The performance on the real-world city map graphs is summarized in Table 2. For all four cities, the exact ILP solver failed to converge within 24 hours. In fact, due to the early termination, the solver returned feasible solutions with significantly higher costs than the heuristic methods in several cases, e.g., Paris. Therefore, we cannot show the optimality ratio on these graphs. Among the baselines, RECURSIVEUFL remains the strongest competitor. Our MPNN architecture achieves the lowest total cost across all four datasets. For instance, on the Paris dataset, the GNN reduces the total cost by approximately 41% compared to the solver’s incumbent solution.

Size generalization (Q2) To answer **Q2**, we test the model

Table 2. Performance comparison on real-world graphs. Baseline results are averaged over 1000 samples, while MPNN results show the mean $\pm$ standard deviation over five training runs.

Candidate	Cost	Paris	London	Shanghai	LA
Solver	Open	60046	247059	90967	116398
	Con.	1940.919	1649.157	548.949	1266.352
	Total	61986.919	248708.157	91515.949	117664.352
	Time	24h	24h	24h	24h
SimpleUFL	Open	60740.267	334712.636	104773.305	137612.179
	Con.	508.156	983.809	386.4197	864.918
	Total	61248.423	335696.445	105159.725	138477.097
	Time	0.001	0.004	0.001	0.002
RecursiveUFL	Open	45421.176	255379.607	79582.307	104533.533
	Con.	779.963	1525.648	583.267	1326.325
	Total	46201.139	256905.256	80165.574	105859.858
	Time	0.019	0.022	0.017	0.017
$O(1)$ -UFL	Open	49408.523	271161.191	85041.974	111659.400
	Con.	675.359	1386.709	525.932	1191.649
	Total	50083.882	272547.900	85567.906	112851.049
	Time	0.067	0.379	0.113	0.146
MPNN	Open	35909.008 $\pm$ 280.415	185331.256 $\pm$ 607.447	60728.737 $\pm$ 447.568	77842.859 $\pm$ 317.723
	Con.	779.735 $\pm$ 6.160	2160.849 $\pm$ 19.504	801.318 $\pm$ 11.597	1757.799 $\pm$ 6.267
	Total	36688.742 $\pm$ 281.216	187492.106 $\pm$ 591.426	61530.055 $\pm$ 436.448	79600.658 $\pm$ 314.253
	Time	0.078 $\pm$ 0.000	0.390 $\pm$ 0.000	0.125 $\pm$ 0.001	0.166 $\pm$ 0.000

trained on 1000-vertex graphs on larger instances up to 10 000 vertices without retraining. As shown in Table 3 and Table 10, the approach exhibits remarkable robustness. The optimality ratio remains nearly constant, i.e., on 2D space increasing merely from 1.009 on the training size to 1.012 on graphs $10\times$ larger, and on 10D space 1.002 to 1.003. This indicates that the MPNN successfully learns the data distribution and generalizes effectively to larger-scale instances.

Table 3. Results on size generalization, shown in mean $\pm$ standard deviations over five runs. MPNN architectures are trained on Geo-1000-10 and tested on larger instances of 10D Euclidean space.

Candidate	Cost	Geo-1000-10	Geo-2000-10	Geo-3000-10	Geo-5000-10	Geo-10000-10
Solver	Open	291.488	638.320	1023.900	1914.170	3961.250
	Con.	520.611	1020.747	1500.204	2382.913	4681.036
	Total	812.099	1659.067	2524.104	4297.083	8642.286
	Time	0.689	2.675	5.158	12.053	55.200
MPNN	Open	298.689 $\pm$ 1.334	653.247 $\pm$ 2.835	1050.268 $\pm$ 4.658	1987.857 $\pm$ 16.523	4106.202 $\pm$ 33.508
	Con.	515.191 $\pm$ 1.301	1009.599 $\pm$ 2.807	1479.893 $\pm$ 4.669	2322.339 $\pm$ 15.082	4563.699 $\pm$ 31.378
	Total	813.879 $\pm$ 0.048	1662.847 $\pm$ 0.135	2530.162 $\pm$ 0.141	4310.197 $\pm$ 1.452	8669.901 $\pm$ 2.278
	Time	0.004 $\pm$ 0.000	0.006 $\pm$ 0.000	0.006 $\pm$ 0.000	0.007 $\pm$ 0.000	0.009 $\pm$ 0.000
	Ratio	1.002 $\pm$ 0.000	1.002 $\pm$ 0.000	1.002 $\pm$ 0.000	1.003 $\pm$ 0.000	1.003 $\pm$ 0.000

In addition to generalizing to larger sizes, we conducted additional experiments to evaluate how our MPNN generalizes to graphs with varying average node degrees. We took the model pretrained on our Geo-1000-2 dataset and evaluated it without any fine-tuning on three new test sets featuring increasingly dense neighborhoods, Geo-1000-2-d10, Geo-1000-2-d12, and Geo-1000-2-d17, where the postfix denotes the average node degree. The results are as follows in Table 4:

As the results show, generalizing across varying node degrees is more challenging than scaling to larger graph sizes. While the inference time remains fast and stable (around 0.004s), the optimality gap gradually widens from 1.002 on the training data distribution to 1.124 on the densest test set. This behavior intuitively makes sense: increasing the graph size while holding the average degree constant preserves the local neighborhood structure, allowing the MPNN to rely on familiar local patterns. In contrast, increasing the average

Table 4. Results on size generalization, shown in mean $\pm$ standard deviations over five runs. MPNN architectures are trained on Geo-1000-2 and tested on higher-degree graphs.

Candidate	Cost	Geo-1000-2	Geo-1000-2-d10	Geo-1000-2-d12	Geo-1000-2-d17
Solver	Open	366.302	297.120	261.190	225.340
	Con.	279.827	282.319	273.323	255.202
	Total	646.129	579.439	534.513	480.542
	Time	0.263	0.540	0.748	1.158
MPNN	Open	380.816 $\pm$ 0.761	295.560 $\pm$ 2.069	253.666 $\pm$ 4.807	225.457 $\pm$ 4.101
	Con.	271.317 $\pm$ 0.735	290.288 $\pm$ 1.995	294.991 $\pm$ 4.249	314.785 $\pm$ 11.337
	Total	652.133 $\pm$ 0.025	585.848 $\pm$ 0.075	548.657 $\pm$ 0.785	540.243 $\pm$ 8.703
	Time	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.005 $\pm$ 0.000
	Ratio	1.009 $\pm$ 0.000	1.011 $\pm$ 0.000	1.026 $\pm$ 0.001	1.124 $\pm$ 0.018

node degree fundamentally alters the structural topology, shifting the distribution of the messages aggregated at each node during inference. This observation aligns directly with recent theoretical work [Levin et al. \(2025\)](#).

Drawing inspiration from recent graph foundation models [Zhao et al. \(2024\)](#); [Beaini et al. \(2023\)](#), pre-training an MPNN on massive, structurally diverse datasets could yield better generalization across varying topologies without re-training. We conducted further experiments using a structurally diverse training set. We generated 10 000 mixed samples (800–1200 nodes, with average degrees ranging from 5 to 15), matching our original training data budget used in the other experiments. Evaluating this new pretrained model on our size-generalization in Table 10 yields the following results in Table 5:

Table 5. Results on size generalization, shown in mean $\pm$ standard deviations over five runs. MPNN architectures are trained on a mixed dataset and tested on larger instances of 2D Euclidean space.

Candidate	Cost	Geo-1000-2	Geo-2000-2	Geo-3000-2	Geo-5000-2	Geo-10000-2
Solver	Open	366.302	745.330	1201.360	1989.970	3980.100
	Con.	279.827	548.501	813.603	1346.195	2677.519
	Total	646.129	1293.831	2014.963	3336.165	6657.619
	Time	0.263	0.831	1.329	3.630	13.971
MPNN	Open	385.499 $\pm$ 0.351	783.525 $\pm$ 0.616	1269.237 $\pm$ 0.564	2101.198 $\pm$ 1.773	4202.280 $\pm$ 3.900
	Con.	267.098 $\pm$ 0.280	523.752 $\pm$ 0.546	769.752 $\pm$ 0.415	1274.294 $\pm$ 1.519	2535.738 $\pm$ 5.384
	Total	652.598 $\pm$ 0.072	1307.277 $\pm$ 0.069	2038.990 $\pm$ 0.149	3375.493 $\pm$ 0.254	6738.018 $\pm$ 0.516
	Time	0.004 $\pm$ 0.000	0.005 $\pm$ 0.000	0.006 $\pm$ 0.000	0.006 $\pm$ 0.000	0.008 $\pm$ 0.000
	Ratio	1.008 $\pm$ 0.000	1.010 $\pm$ 0.000	1.011 $\pm$ 0.000	1.011 $\pm$ 0.000	1.012 $\pm$ 0.000

This demonstrates that training on a mixed dataset perfectly preserves the excellent size-generalization capabilities seen in Table 10.

Crucially, we also evaluated this mixed-trained model on Geo-1000-2-d22 (average degree 22) and Geo-1000-2-d27 (degree 27), and the results are in Table 6. As shown, the mixed training strategy is highly effective: the optimality ratio remains exceptionally stable (1.006 to 1.020) even on higher densities, successfully mitigating the structural generalization degradation observed in our previous single-distribution model.

Comparison with k -Means and k -Medoids (Q3) To answer Q3, we evaluate the adaptability of our approach by modifying the MPNN loss to minimize the sum of squared distances, effectively simulating the clustering objective. We

Table 6. Results on size generalization, shown in mean $\pm$ standard deviations over five runs. MPNN architectures are trained on a mixed dataset and tested on higher-degree graphs.

Candidate	Cost	Geo-1000-2-d10	Geo-1000-2-d12	Geo-1000-2-d17	Geo-1000-2-d22	Geo-1000-2-d27
Solver	Open	297.120	261.190	225.340	205.900	191.320
	Con.	282.319	273.323	255.202	243.163	233.445
	Total	579.439	534.513	480.542	449.063	424.765
	Time	0.540	0.748	1.158	1.558	1.854
MPNN	Open	305.514 $\pm$ 1.009	263.706 $\pm$ 0.406	220.253 $\pm$ 0.763	194.649 $\pm$ 0.643	176.500 $\pm$ 0.233
	Con.	277.809 $\pm$ 0.999	273.959 $\pm$ 0.373	264.343 $\pm$ 0.690	260.615 $\pm$ 0.371	259.011 $\pm$ 1.102
	Total	583.324 $\pm$ 0.010	537.666 $\pm$ 0.033	484.596 $\pm$ 0.072	455.264 $\pm$ 0.271	435.512 $\pm$ 0.869
	Time	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.005 $\pm$ 0.000	0.005 $\pm$ 0.001	0.005 $\pm$ 0.000
	Ratio	1.006 $\pm$ 0.000	1.006 $\pm$ 0.000	1.008 $\pm$ 0.000	1.013 $\pm$ 0.001	1.020 $\pm$ 0.001

compare this variant against: (i) Standard k -Means++ and k -Medoids++ initialization; (ii) Use the MPNN-predicted facility locations as initialization for k -Means and k -Medoids. The results in Table 7 reveal a consistent hierarchy in connection cost (inertia): k -Means++ < GNN < k -Medoids++. The advantage of k -Means++ is well known: it minimizes inertia by placing centroids in an arbitrary continuous space, thereby enabling solutions that are not valid for the facility location problem. However, in the discrete setting where both methods are constrained to place centroids on existing nodes, our MPNN approach consistently outperforms k -Medoids++. Moreover, when initialized with MPNN-predicted locations, both k -Means and k -Medoids have shown improvements in inertia in most cases. This confirms that our framework effectively adapts to clustering objectives.

Table 7. Comparison between MPNN trained with squared connection cost, k -Means, and k -Medoids. Results are shown as mean $\pm$ standard deviation across five runs trained on each dataset.

Candidate	Cost	Geo-1000-2	Geo-1000-5	Geo-1000-10
GNN + SimpleUFL	Open	365.167 $\pm$ 1.180	359.636 $\pm$ 1.272	260.530 $\pm$ 2.172
	Con.	162.800 $\pm$ 1.156	296.984 $\pm$ 1.938	444.445 $\pm$ 3.195
	Time	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000	0.004 $\pm$ 0.000
k -Means++	Inertia	130.161 $\pm$ 0.947	223.515 $\pm$ 1.038	323.422 $\pm$ 1.886
	Time	0.042 $\pm$ 0.001	0.041 $\pm$ 0.000	0.055 $\pm$ 0.000
k -Means (MPNN init)	Inertia	132.438 $\pm$ 1.077	213.991 $\pm$ 0.353	294.124 $\pm$ 0.186
	Time	0.007 $\pm$ 0.001	0.007 $\pm$ 0.000	0.013 $\pm$ 0.001
k -Medoids++	Inertia	170.904 $\pm$ 1.258	312.669 $\pm$ 1.449	458.796 $\pm$ 2.592
	Time	0.028 $\pm$ 0.000	0.026 $\pm$ 0.000	0.017 $\pm$ 0.000
k -Medoids (MPNN init)	Inertia	161.600 $\pm$ 1.148	280.005 $\pm$ 0.321	398.592 $\pm$ 2.074
	Time	0.095 $\pm$ 0.002	0.095 $\pm$ 0.001	0.081 $\pm$ 0.001

5. Limitations and future directions

While our MPNN frameworks devise the first *unsupervised* neural architecture for the UniFL, several limitations and open problems remain. That is, our approach, which focuses on the UniFL and inherits structural assumptions from the underlying distributed approximation algorithm. While this is essential for preserving worst-case guarantees, it may also restrict flexibility, i.e., the architecture is tailored to UniFL and may require non-trivial redesign to extend to richer variants such as non-uniform opening costs, capacitated facility location, or settings with additional side constraints.

Looking forward, there are several promising directions for future research. First, extending the framework beyond UniFL—e.g., to metric facility location with non-uniform costs, capacitated variants, or related problems such as k -median or k -center—would broaden its applicability and test its generality, ultimately characterizing the set of problems for which MPNN architectures can learn constant-factor approximations. Secondly, developing tighter theoretical analyses that connect learned message passing to instance-dependent approximation behavior, together with understanding under which conditions variants of gradient descent can successfully minimize the expected loss, would help explain when and why training improves solution quality.

6. Conclusion

Here, we introduced an unsupervised, light-weight, and fully differentiable MPNN architecture for UniFL. By mirroring a distributed approximation procedure—using learned messages to estimate informative “radii” and a probabilistic facility-opening mechanism—the architecture can be trained end-to-end without relying on optimal labels or reinforcement learning, while still providing worst-case approximation guarantees. Moreover, our framework provides provable size generalization, i.e., parameters learned on smaller instances transfer to arbitrarily larger graphs at test time. Across experiments, the method improves on standard approximation baselines on natural input distributions and narrows the gap to computationally heavier exact approaches, while remaining scalable. *Overall, our results show that leveraging classical algorithmic ideas can yield differentiable algorithms with guarantees, opening the door to a broader class of problems.*

Impact statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgments

CM and CQ are partially funded by a DFG Emmy Noether grant (468502433) and RWTH Junior Principal Investigator Fellowship under Germany’s Excellence Strategy. SJ acknowledges funding from the Alexander von Humboldt Foundation.

References

- Ahmed, K., Zeng, Z., Niepert, M., and den Broeck, G. V. SIMPLE: A gradient estimator for k-subset sampling. In *ICLR*, 2023.
- Almanza, M., Chierichetti, F., Lattanzi, S., Panconesi, A., and Re, G. Online facility location with multiple advice. *NeurIPS*, 34:4661–4673, 2021.
- Arya, V., Garg, N., Khandekar, R., Meyerson, A., Munagala, K., and Pandit, V. Local Search Heuristics for k -Median and Facility Location Problems. In *SIAM Journal on Computing*, volume 33, pp. 544–562, 2004.
- Azizian, W. and Lelarge, M. Characterizing the expressive power of invariant and equivariant graph neural networks. In *ICLR*, 2021.
- Bădoiu, M., Czumaj, A., Indyk, P., and Sohler, C. Facility location in sublinear time. In *ICALP*, pp. 866–877, 2005.
- Bairi, R. B., Iyer, R., Ramakrishnan, G., and Bilmes, J. Summarization of multi-document topic hierarchies using submodular mixtures. In *Association of Computational Linguists (ACL)*, 2015.
- Baskin, I. I., Palyulin, V. A., and Zefirov, N. S. A neural device for searching direct arxiv preprintelations between structures and properties of chemical compounds. *Journal of Chemical Information and Computer Sciences*, 37(4): 715–721, 1997.
- Beaini, D., Huang, S., Cunha, J. A., Li, Z., Moisesescu-Pareja, G., Dymov, O., Maddrell-Mander, S., McLean, C., Wenkel, F., Müller, L., et al. Towards foundational models for molecular learning on large-scale multi-task datasets. *arXiv preprint arXiv:2310.04292*, 2023.
- Bello, I., Pham, H., Le, Q. V., Norouzi, M., and Bengio, S. Neural combinatorial optimization with reinforcement learning. *arXiv preprint arXiv:1611.09940*, 2016.
- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Berns, A., Hegeman, J., and Pemmaraju, S. V. Super-fast distributed algorithms for metric facility location. In *ICALP*. Springer, 2012.
- Bruna, J., Zaremba, W., Szlam, A., and LeCun, Y. Spectral networks and deep locally connected networks on graphs. In *International Conference on Learning Representation*, 2014.
- Böker, J., Levie, R., Huang, N., Villar, S., and Morris, C. Fine-grained expressivity of graph neural networks. In *NeurIPS*, 2023.
- Cai, J., Fürer, M., and Immerman, N. An optimal lower bound on the number of variables for graph identifications. *Combinatorica*, 12(4):389–410, 1992.
- Cappart, Q., Chételat, D., Khalil, E. B., Lodi, A., Morris, C., and Veličković, P. Combinatorial optimization and reasoning with graph neural networks. *JMLR*, 2023.
- Charikar, M. and Guha, S. Improved combinatorial algorithms for the facility location and k-median problems. In *STOC*, 1999.
- Chen, Z., Villar, S., Chen, L., and Bruna, J. On the equivalence between graph isomorphism testing and function approximation with GNNs. In *NeurIPS*, pp. 15868–15876, 2019.
- Chen, Z., Liu, J., Wang, X., Lu, J., and Yin, W. On representing linear programs by graph neural networks. In *ICLR*, 2023.
- de Luca, A. B. and Fountoulakis, K. Simulation of graph algorithms with looped transformers. In *ICML*, 2024.
- Defferrard, M., Bresson, X., and Vandergheynst, P. Convolutional neural networks on graphs with fast localized spectral filtering. In *NeurIPS*, 2016.
- Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gómez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A., and Adams, R. P. Convolutional networks on graphs for learning molecular fingerprints. In *NeurIPS*, 2015.
- Fotakis, D., Gergatsouli, E., Gouleakis, T., Patrís, N., and Toliás, T. Improved bounds for online facility location with predictions. In *AAAI*, 2025.
- Gama, F., Marques, A. G., Leus, G., and Ribeiro, A. Convolutional neural network architectures for signals supported on graphs. *IEEE Transactions on Signal Processing*, 67(4):1034–1049, 2019.
- Garg, V. K., Jegelka, S., and Jaakkola, T. S. Generalization and representational limits of graph neural networks. In *ICLR*, 2020.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., and Lodi, A. Exact combinatorial optimization with graph convolutional neural networks. In *NeurIPS*, 2019.
- Geerts, F. and Reutter, J. L. Expressiveness and approximation properties of graph neural networks. In *ICLR*, 2022.
- Gehweiler, J., Lammersen, C., and Sohler, C. A distributed $o(1)$ -approximation algorithm for the uniform facility location problem. *Algorithmica*, 68(3):643–670, 2014.

- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., and Dahl, G. E. Neural message passing for quantum chemistry. In *ICLR*, 2017.
- Goller, C. and Küchler, A. Learning task-dependent distributed representations by backpropagation through structure. In *ICNN*, 1996.
- Guha, S. and Khuller, S. Greedy strikes back: Improved facility location algorithms. *J. Algorithms*, 31(1):228–248, 1999.
- Guo, X., Li, A., Wang, Y., Jegelka, S., and Wang, Y. G1: Teaching llms to reason on graphs with reinforcement learning. In *NeurIPS (NeurIPS)*, 2025.
- Gupta, A. and Tangwongsan, K. Simpler Analyses of Local Search Algorithms for Facility Location. In *arXiv preprint arXiv:0809.2554*, 2008.
- Hamilton, W. L., Ying, Z., and Leskovec, J. Inductive representation learning on large graphs. In *NeurIPS*, 2017.
- He, Y. and Vitercik, E. Primal-dual neural algorithmic reasoning. *arXiv preprint arXiv:2505.24067*, 2025.
- Jain, K. and Vazirani, V. V. Approximation algorithms for metric facility location and k -Median problems using the primal-dual schema and Lagrangian relaxation. In *Journal of the ACM*, volume 48, pp. 274–296, 2001. doi: 10.1145/375827.375845.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel–softmax. In *ICLR*, 2017.
- Jiang, S. H., Liu, E., Lyu, Y., Tang, Z. G., and Zhang, Y. Online facility location with predictions. In *ICLR*, 2022.
- Karalias, N. and Loukas, A. Erdos goes neural: an unsupervised learning framework for combinatorial optimization on graphs. In *NeurIPS*, 2020.
- Karalias, N., Robinson, J., Loukas, A., and Jegelka, S. Neural set function extensions: Learning with discrete functions in high dimensions. In *NeurIPS (NeurIPS)*, 2022.
- Karalias, N., Rafiey, A., Xu, Y., Luo, Z., Tahmasebi, B., Jiang, C., and Jegelka, S. Geometric algorithms for neural combinatorial optimization with constraints. *arXiv preprint arxiv:2510.24039*, 2025.
- Keriven, N., Bietti, A., and Vaiter, S. Convergence and stability of graph convolutional networks on large random graphs. In *NeurIPS*, 2020.
- Khalil, E. B., Dai, H., Zhang, Y., Dilkina, B., and Song, L. Learning combinatorial optimization algorithms over graphs. In *NeurIPS*, 2017.
- Khalil, E. B., Morris, C., and Lodi, A. MIP-GNN: A data-driven framework for guiding combinatorial solvers. In *AAAI*, 2022.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *ICLR*, 2017.
- Kireev, D. B. Chemnet: A novel neural network based method for graph/property mapping. *Journal of Chemical Information and Computer Sciences*, 35(2):175–180, 1995.
- Lammersen, C. *Approximation Techniques for Facility Location and Their Applications in Metric Embeddings*. PhD thesis, Technische Universität Dortmund, 2010. Ph.D. thesis.
- Le, T. and Jegelka, S. Limits, approximation and size transferability for gnns on sparse graphs via graphops. In *NeurIPS*, 2023.
- Levie, R., Monti, F., Bresson, X., and Bronstein, M. M. Cayleynets: Graph convolutional neural networks with complex rational spectral filters. *IEEE Transactions on Signal Processing*, 67(1):97–109, 2019.
- Levie, R., Huang, W., Bucci, L., Bronstein, M., and Kutylniok, G. Transferability of spectral graph convolutional neural networks. *Journal of Machine Learning Research*, 22(1), January 2021. ISSN 1532-4435.
- Levin, E., Ma, Y., Díaz, M., and Villar, S. On transferring transferability: Towards a theory for size generalization. *arXiv preprint arXiv:2505.23599*, 2025.
- Liang, H., Borde, H. S. d. O., Sripathmanathan, B., Bronstein, M., and Dong, X. Towards quantifying long-range interactions in graph machine learning: a large graph dataset and a measurement. *arXiv preprint arXiv:2503.09008*, 2025.
- Maddison, C. J., Mnih, A., and Teh, Y. W. The concrete distribution: A continuous relaxation of discrete random variables. In *ICLR*, 2017.
- Maehara, T. and NT, H. A simple proof of the universality of invariant/equivariant graph neural networks. *arXiv preprint arXiv:1910.03802*, 2019.
- Mahdian, M., Ye, Y., and Zhang, J. Approximation algorithms for metric facility location problems. *SIAM Journal on Computing*, 36(2):411–432, 2006. doi: 10.1137/S0097539703435716.
- Mahdian, M., Nazerzadeh, H., and Saberi, A. Online optimization with uncertain information. *ACM Transactions on Algorithms (TALG)*, 8(1):1–29, 2012.

- Maskey, S., Levie, R., and Kutyniok, G. Transferability of graph neural networks: An extended graphon approach. *Applied and Computational Harmonic Analysis*, 63:48–83, 2023.
- Merkwirth, C. and Lengauer, T. Automatic generation of complementary descriptors with molecular graph networks. *Journal of Chemical Information and Modeling*, 45(5):1159–1168, 2005.
- Mettu, R. R. and Plaxton, C. G. The Online Median Problem. In *SIAM Journal on Computing*, volume 32, pp. 816–832, 2003.
- Meyerson, A. Online facility location. In *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, pp. 426–431. IEEE, 2001.
- Micheli, A. Neural network for graphs: A contextual constructive approach. *IEEE Transactions on Neural Networks*, 20(3):498–511, 2009.
- Micheli, A. and Sestito, A. S. A new neural network model for contextual processing of graphs. In *Italian Workshop on Neural Nets Neural Nets and International Workshop on Natural and Artificial Immune Systems*, pp. 10–17, 2005.
- Mitzenmacher, M. and Vassilvitskii, S. Algorithms with predictions. In Roughgarden, T. (ed.), *Beyond the Worst-Case Analysis of Algorithms*, pp. 646–662. Cambridge University Press, 2020.
- Monti, F., Boscaini, D., Masci, J., Rodolà, E., Svoboda, J., and Bronstein, M. M. Geometric deep learning on graphs and manifolds using mixture model cnns. In *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5425–5434, 2017.
- Morris, C., Ritzert, M., Fey, M., Hamilton, W. L., Lenssen, J. E., Rattan, G., and Grohe, M. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.
- Morris, C., Lipman, Y., Maron, H., Rieck, B., Kriege, N. M., Grohe, M., Fey, M., and Borgwardt, K. Weisfeiler and leman go machine learning: The story so far. *Journal of Machine Learning Research*, 24(333):1–59, 2023.
- Moscibroda, T. and Wattenhofer, R. Facility location: distributed approximation. In *PODC*, 2005.
- Müller, L., Galkin, M., Morris, C., and Rampásek, L. Attending to graph transformers. *TMLR*, 2024.
- Nerem, R. R., Chen, S., Dasgupta, S., and Wang, Y. Graph neural networks extrapolate out-of-distribution for shortest paths. *arXiv preprint arXiv:2503.19173*, 2025.
- Niepert, M., Minervini, P., and Franceschi, L. Implicit MLE: Backpropagating through discrete exponential family distributions. In *NeurIPS*, 2021.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- Perron, L. and Furnon, V. Or-tools. URL <https://developers.google.com/optimization/>.
- Petersen, F., Borgelt, C., Mishra, A. A., and Ermon, S. Generalizing stochastic smoothing for differentiation and gradient estimation. *arXiv arXiv:2410.08125*, 2024.
- Qian, C., Chételat, D., and Morris, C. Exploring the power of graph neural networks in solving linear optimization problems. In *AISTATS*, 2024.
- Rauchwerger, L., Jegelka, S., and Levie, R. Generalization, expressivity, and universality of graph neural networks on attributed graphs. In *ICLR*, 2025.
- Ruiz, L., Chamon, L., and Ribeiro, A. Graphon neural networks and the transferability of graph neural networks. In *NeurIPS*, 2020.
- Sanford, C., Fatemi, B., Hall, E., Tsitsulin, A., Kazemi, M., Halcrow, J., Perozzi, B., and Mirrokni, V. Understanding transformer reasoning capabilities via graph algorithms. In *NeurIPS*, 2024a.
- Sanford, C., Hsu, D., and Telgarsky, M. Transformers, parallel computation, and logarithmic depth. In *ICML*, 2024b.
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The graph neural network model. *IEEE Transactions on Neural Networks*, 20(1):61–80, 2009.
- Scarselli, F., Tsoi, A. C., and Hagenbuchner, M. The Vapnik-Chervonenkis dimension of graph and recursive neural networks. *Neural Networks*, pp. 248–259, 2018.
- Scavuzzo, L., Aardal, K. I., Lodi, A., and Yorke-Smith, N. Machine learning augmented branch and bound for mixed integer linear programming. *Mathematical Programming (Series B)*, 2024.
- Selsam, D. and Bjørner, N. S. Neurocore: Guiding high-performance SAT solvers with unsat-core predictions. *arXiv preprint arXiv:1903.04671*, 2019.
- Selsam, D., Lamm, M., Bünz, B., Liang, P., de Moura, L., and Dill, D. L. Learning a SAT solver from single-bit supervision. In *ICLR*, 2019.

- Shmoys, D. B. Approximation algorithms for facility location problems. In Hochbaum, D. (ed.), *Approximation Algorithms for NP-hard Problems*. PWS, 2000.
- Shmoys, D. B., Tardos, É., and Aardal, K. Approximation algorithms for facility location problems. In *Proceedings of the twenty-ninth annual ACM symposium on Theory of computing*, pp. 265–274, 1997.
- Sperduti, A. and Starita, A. Supervised neural networks for the classification of structures. *IEEE Transactions on Neural Networks*, 8(3):714–35, 1997.
- Thorup, M. Quick k-Median, k-Center, and Facility Location for Sparse Graphs. In *SIAM Journal on Computing*, volume 34, pp. 405–432, 2004.
- Tönshoff, J. and Grohe, M. Learning from algorithm feedback: One-shot SAT solver guidance with gnns. *arXiv preprint arXiv:2505.16053*, 2025.
- Vasileiou, A., Finkelshtein, B., Geerts, F., Levie, R., and Morris, C. Covered forest: Fine-grained generalization analysis of graph neural networks. In *ICML*, 2025a.
- Vasileiou, A., Jegelka, S., Levie, R., and Morris, C. Survey on generalization theory for graph neural networks. *arXiv preprint arXiv:2503.15650*, 2025b.
- Vasileiou, A., Stoll, T., and Morris, C. Understanding generalization in node and link prediction. *CoRR*, abs/2507.00927, 2025c.
- Vazirani, V. V. *Approximation algorithms*, volume 1. Springer, Berlin, Heidelberg, 2001. ISBN 978-3-540-65367-7.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y. Graph attention networks. In *ICLR*, 2018.
- Wang, W., Hu, Y., Tiwari, M., Khurshid, S., McMillan, K., and Mikkilainen, R. NeuroComb: Improving sat solving with graph neural networks. In *ICLR*, 2024.
- Wei, K., Liu, Y., Kirchhoff, K., and Bilmes, J. Unsupervised submodular subset selection for speech data. In *IEEE ICASSP*, 2014.
- Weisfeiler, B. and Leman, A. The reduction of a graph to canonical form and the algebra which appears therein. *nti, Series*, 2(9):12–16, 1968.
- Wittig, S., Vasileiou, A., Nerem, R. R., Stoll, T., Geerts, F., Wang, Y., and Morris, C. Which algorithms can graph neural networks learn? In *ICML*, 2026.
- Xu, K., Hu, W., Leskovec, J., and Jegelka, S. How powerful are graph neural networks? In *ICLR*, 2019.
- Xu, K., Zhang, M., Li, J., Du, S. S., Kawarabayashi, K.-I., and Jegelka, S. How neural networks extrapolate: From feedforward to graph neural networks. In *ICLR*, 2021.
- Yau, M., Karalias, N., Lu, E., Xu, J., and Jegelka, S. Are graph neural networks optimal approximation algorithms? In *NeurIPS*, 2024.
- Yehudai, G., Fetaya, E., Meir, E., Chechik, G., and Maron, H. From local structures to size generalization in graph neural networks. In *ICML*, 2021.
- Yehudai, G., Sanford, C., Bechler-Speicher, M., Fischer, O., Gilad-Bachrach, R., and Globerson, A. Depth-width trade-offs in algorithmic reasoning of graph tasks with transformers. *arXiv preprint arXiv:2503.01805*, 2025.
- Zhao, J., Zhu, Z., Galkin, M., Mostafa, H., Bronstein, M., and Tang, J. Fully-inductive node classification on arbitrary graphs. *arXiv preprint arXiv:2405.20445*, 2024.

A. Extended background

Here, we introduce additional background material.

Continuity on metric spaces Let $(\mathcal{X}, d_{\mathcal{X}})$ and $(\mathcal{Y}, d_{\mathcal{Y}})$ be two pseudo-metric spaces. A function $f: \mathcal{X} \rightarrow \mathcal{Y}$ is called *c_f-Lipschitz continuous* if, for $x, x' \in \mathcal{X}$,

$$d_{\mathcal{Y}}(f(x), f(x')) \leq c_f \cdot d_{\mathcal{X}}(x, x').$$

Message-passing graph neural networks Message-passing graph neural networks (MPNNs) learn a d -dimensional real-valued vector of each vertex in a graph by aggregating information from neighboring vertices. Following Gilmer et al. (2017), let G be an attributed, edge-weighted graph with initial vertex feature $\mathbf{h}_v^{(0)} \in \mathbb{R}^{1 \times d_0}$, $d_0 \in \mathbb{N}$, for $v \in V(G)$. An *MPNN architecture* consists of a composition of L neural network layers for some $L > 0$. In each *layer*, $t \in \mathbb{N}$, we compute a *vertex feature*

$$\mathbf{h}_v^{(t)} := \text{UPD}^{(t)}\left(\mathbf{h}_v^{(t-1)}, \text{AGG}^{(t)}\left(\{\{\mathbf{h}_v^{(t-1)}, \mathbf{h}_u^{(t-1)}, w_G(v, u)\} \mid u \in N(v)\}\}\right)\right) \in \mathbb{R}^{1 \times d_t}, \quad (6)$$

$d_t \in \mathbb{N}$, for $v \in V(G)$, where $\text{UPD}^{(t)}$ and $\text{AGG}^{(t)}$ may be parameterized functions, e.g., neural networks. In the case of graph-level tasks, e.g., graph classification, one uses a *readout*, where

$$\mathbf{h}_G := \text{READOUT}\left(\{\{\mathbf{h}_v^{(L)} \mid v \in V(G)\}\}\right) \in \mathbb{R}^{1 \times d},$$

to compute a single vectorial representation based on learned vertex features after iteration L . Again, READOUT may be a parameterized function.

Feedforward neural networks An L -layer *feed-forward neural network* (FNN), for $L \in \mathbb{N}$, is a parametric function $\text{FNN}_{\theta}^{(L)}: \mathbb{R}^{1 \times d_0} \rightarrow \mathbb{R}^{1 \times d}$, $d_0, d > 0$, where $\theta := (\mathbf{W}^{(1)}, \mathbf{b}^{(1)}, \dots, \mathbf{W}^{(L)}, \mathbf{b}^{(L)}) \in \Theta$, $\mathbf{W}^{(1)} \in \mathbb{R}^{d_0 \times d}$, $\mathbf{W}^{(i)} \in \mathbb{R}^{d \times d}$, $\mathbf{b}^{(i)} \in \mathbb{R}^{1 \times d}$, for $i \in [L]$, and Θ is an appropriately chosen set, where

$$\mathbf{x} \mapsto \sigma\left(\dots \sigma\left(\sigma\left(\mathbf{x}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}\right)\mathbf{W}^{(2)} + \mathbf{b}^{(2)}\right) \dots \mathbf{W}^{(L)} + \mathbf{b}^{(L)}\right) \in \mathbb{R}^{1 \times d},$$

for $\mathbf{x} \in \mathbb{R}^{1 \times d}$. Here, the function $\sigma: \mathbb{R} \rightarrow \mathbb{R}$ is an *activation function*, applied point-wisely, e.g., a *rectified linear unit* (ReLU), where $\sigma(x) := \max(0, x)$. We define

$$\text{FNN}_{L, d_0, d} := \left\{ \text{FNN}_{\theta}^{(L)}: \mathbb{R}^{1 \times d_0} \rightarrow \mathbb{R}^{1 \times d} \mid \theta \in \Theta \right\}.$$

By default, we assume that activation functions are always ReLU unless otherwise stated. Given a function $f: \mathbb{R}^{1 \times d_0} \rightarrow \mathbb{R}^{1 \times d}$, we write $f \equiv \text{FNN}_{L, d_0, d}$ if there exists an $L \in \mathbb{N}$ and an $f_{\theta} \in \text{FNN}_{L, d_0, d}$ such that $f(x) = f_{\theta}(x)$, for $x \in \mathbb{R}^{1 \times d_0}$.

B. Extended related work

Here, we discuss more related work.

Expressivity and generalization abilities of MPNNs The expressivity of an MPNN refers to its ability to express or approximate a broad class of functions over graphs. High expressivity means that the network can represent many such functions. In the literature, MPNN expressivity is typically analyzed from two complementary perspectives, i.e., algorithmic alignment with graph isomorphism tests (Morris et al., 2023) and universal approximation theorems (Azizian & Lelarge, 2021; Böker et al., 2023; Geerts & Reutter, 2022). Works in the first line of research investigate whether an MPNN, with suitably chosen parameters, can distinguish the same pairs of non-isomorphic graphs as the 1-dimensional Weisfeiler–Leman algorithm (Weisfeiler & Leman, 1968) or its more powerful generalization (Cai et al., 1992). An MPNN is said to distinguish two non-isomorphic graphs if it computes different vector-valued representations for them. Foundational results by Morris et al. (2019) and Xu et al. (2019) established that the expressive power of any MPNN is upper-bounded by that of 1-dimensional Weisfeiler–Leman algorithm. The second line of research examines which functions over the domain of graphs can be approximated arbitrarily well by MPNNs (Azizian & Lelarge, 2021; Böker et al., 2023; Chen et al., 2019; Geerts & Reutter, 2022; Maehara & NT, 2019). Compared to expressivity, there is comparatively little work

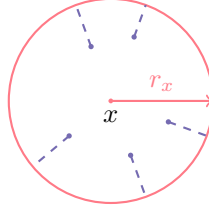


Figure 2. Illustration of the radius r_x of the element x according to Equation (2), where the lengths of the dashed lines are summed up.

on the generalization abilities of MPNNs, e.g., see Garg et al. (2020); Scarselli et al. (2018); Vasileiou et al. (2025a;c). In addition, there is a growing body of work on the size-generalization abilities of MPNNs, e.g., (Levin et al., 2025; Maskey et al., 2023; Xu et al., 2021; Le & Jegelka, 2023; Ruiz et al., 2020; Keriven et al., 2020; Levie et al., 2021; Yehudai et al., 2021); see Vasileiou et al. (2025b) for a detailed survey on MPNNs’ generalization capabilities.

(MP)NNs for CO As mentioned above, there is a growing body of work on using MPNNs for CO, either for directly solving CO problems heuristically, e.g. (Bello et al., 2016; Karalias & Loukas, 2020; Khalil et al., 2017; Selsam et al., 2019) or by integrating them into exact solvers, e.g., in the context of integer-linear optimization (Scavuzzo et al., 2024) or SAT solving (Selsam & Bjørner, 2019; Tönshoff & Grohe, 2025; Wang et al., 2024); see Cappart et al. (2023) for an overview. Recent advances include simulating primal-dual approximation (He & Vitercik, 2025), solving linear optimization problems (Chen et al., 2023; Qian et al., 2024), self-supervised learning for CO problems (Karalias et al., 2025), and provably learning the Bellman–Ford algorithm for shortest paths (Nerem et al., 2025). Recently Yau et al. (2024) showed that MPNN can recover a semidefinite programming-based (optimal) approximation algorithm for maximum constraint-satisfaction problems, such as maximum cut, vertex cover, and maximum 3-SAT. Finally, there is a growing number of works on the capabilities of (graph) transformers (Müller et al., 2024) to execute (graph) algorithms, e.g., de Luca & Fountoulakis (2024); Sanford et al. (2024a;b); Yehudai et al. (2025); Guo et al. (2025). See Cappart et al. (2023) for a survey.

Approximation algorithms with predictions A different approach that aims at connecting approximation algorithms and machine learning is *algorithms with predictions* (Mitzenmacher & Vassilvitskii, 2020). In this framework, one seeks to design (online) algorithms that have access to a machine-learning-based prediction oracle that provides advice of unknown quality. If the advice is bad, the goal is to be as good as the best worst-case online algorithm (denoted *robustness*), and if the advice is good, the goal is to be as good as the advice (denote *consistency*). For the online facility location problem (Meyerson, 2001), several prediction-based algorithms are known (Mahdian et al., 2012; Almanza et al., 2021; Jiang et al., 2022; Fotakis et al., 2025). One challenge with algorithms with prediction is often that it is unclear how the predictions will be learned.

C. Missing proofs

Here, we outline missing proofs from the main paper.

C.1. Approximation factor of SIMPLEUNIFORMFL

Here, we prove the $\mathcal{O}(\log(n))$ -approximation factor of SIMPLEUNIFORMFL.

Lemma 6. *Let $S := (\mathcal{X}, d)$ be a UniFL instance. Let $x \in \mathcal{X}$ be arbitrary. Then there exists a set $T \subseteq \mathcal{X}$ of size k such that for every $y \in T$ we have $d(y, x) \leq 2r_x$ and $k \geq 1/(2r_y)$.*

Proof. Let $x \in \mathcal{X}$ be arbitrary. Define $x^{(0)} = x$ and $x^{(1)} \in B(x^{(0)}, r_{x^{(0)}})$ to be the point with minimum $r_{x^{(1)}}$. If $r_{x^{(1)}} \geq r_{x^{(0)}}/2$ then define $T = B(x, r_{x^{(0)}})$. We know that $k := |B(x, r_{x^{(0)}})| \geq 1/r_{x^{(0)}} \geq 1/(2r_{x^{(1)}}) \geq 1/(2r_y)$, for every $y \in T$ which satisfies the second condition of the lemma while the first condition is clearly satisfied.

Otherwise, $r_{x^{(1)}} < r_{x^{(0)}}/2$. In this case, we define $x^{(2)} \in B(x^{(1)}, r_{x^{(1)}})$ to be the point with minimum $r_{x^{(2)}}$ and, more generally, $x^{(i)} \in B(x^{(i-1)}, r_{x^{(i-1)}})$ to be the point with minimum $r_{x^{(i)}}$. As before, we proceed as follows. If $r_{x^{(i)}} \geq r_{x^{(i-1)}}/2$ then the recursion stops and we define $T = B(x_{(i-1)}, r_{x_{(i-1)}})$. Otherwise, we recurse again.

Observe that the recursion eventually stops as the smallest possible radius is $1/n$. Let $x^{(r)}$ be the point that is considered when the recursion stops, and so $T = B(x_{(r-1)}, r_{x_{(r-1)}})$. We know that $k := |B(x^{(r-1)}, r_{x_{(r-1)}})| \geq 1/r_{x_{(r-1)}} \geq 1/(2r_{x_{(r)}}) \leq$

$1/(2r_y)$ for every $y \in T$ which implies that every $y \in T$ satisfies the second condition of the lemma. Furthermore, for every $y \in T$ we have $d(x, y) \leq d(x^{(k)}, y) + \sum_{i=1}^k d(x^i, x^{i-1}) \leq d(x^{(k)}, y) + r_x \cdot \sum_{i=1}^k \frac{1}{2^{j-1}} \leq r_x/2^{k-1} + \sum_{i=1}^k \frac{1}{2^{j-1}} = 2r_x$. Hence, the first condition is also satisfied, and the lemma follows. $\square$

Lemma 7 (Proposition 2 in the main paper). *Let $S = (\mathcal{X}, d)$ be a UniFL instance, then the algorithm SIMPLEUNIFORMFL with $c \geq 2$ computes a solution to the UniFL for S with expected cost $\mathcal{O}(\log(n)) \cdot \text{OPT}_S$, where OPT_S denotes the cost of an optimal solution.*

Proof. For $x \in \mathcal{X}$ let Y_x denotes the indicator random variable for the event that $x \in F_1$. Furthermore, let $Z_x = \min(1, d(x, F))$ denote the random variable for the connection cost, i.e., the distance to the closest open facility (where we assume that this distance is at most 1, since otherwise we open a facility in F_2). We first show that by Lemma 1

$$\mathbb{E}\left(\sum_{x \in \mathcal{X}} Y_x\right) = \sum_{x \in \mathcal{X}} \mathbb{E}(Y_x) \leq c \cdot \log n \cdot \sum_{x \in \mathcal{X}} r_x \in \mathcal{O}(\log n) \cdot \text{OPT}_S.$$

In order to estimate Z_x we will first argue that with good probability, there is an open facility within distance $2r_x$, and we pay 1 to open a facility in x , if there is no such nearby facility. By Lemma 6 there is a set T of k points such that for every $y \in T$ we have $d(x, y) \leq 2r_x$ and radius $r_y \geq 1/(2k)$. The probability that none of these points has been opened at most $(1 - \min(1, \frac{c \ln n}{2k}))^k$. If $k < \frac{c \ln n}{2}$ this probability is 0, so we assume in the following that $k \geq \frac{c \ln n}{2}$. In this case, we get $(1 - \min(1, \frac{c \ln n}{2k}))^k = (1 - \frac{c \ln n}{2k})^k \leq e^{-k \frac{c \ln n}{2k}} = e^{-\frac{c}{2} \ln n} = \frac{1}{n^{c/2}}$. Thus,

$$\mathbb{E}\left(\sum_{x \in \mathcal{X}} Z_x\right) = \sum_{x \in \mathcal{X}} \mathbb{E}(Z_x) \leq \sum_{x \in \mathcal{X}} \left(2r_x + \frac{1}{n^{c/2}}\right) \in \mathcal{O}(\text{OPT}_S).$$

The lemma follows from the fact that the expected facility location cost is at most the sum of the expected opening cost and the connection cost. $\square$

C.2. Approximation capabilities of the MPNN architecture

Here, we prove the $\mathcal{O}(\log(n))$ -approximation factor of the MPNN architecture.

Proposition 8 (Proposition 3 in the main paper). *For every $n \in \mathbb{N}$, there exists a $k \in \mathbb{N}$, a discretization*

$$0 = a_0 < a_1 < \dots < a_k = 1,$$

and an MPNN architecture $\text{mpnn}_\theta: \mathcal{V}(\mathcal{G}_n) \rightarrow [0,1]$ parameterized by θ , following Section 3.2, and a parameter assignment θ^ such that for every UniFL instance $S := (\mathcal{X}, d)$ with $|\mathcal{X}| = n$, the MPNN mpnn_{θ^*} outputs vertex-wise probabilities*

$$p_v := \text{mpnn}_{\theta^*}(G_S, v) \in [0,1], \quad \text{for } v \in V(G_S),$$

such that

$$\mathbb{E}[\text{cost}(\mathcal{X}, p)] \leq \mathcal{O}(\log(n)) \cdot \text{Opt}_S,$$

where $\text{cost}(X, p)$ denotes the facility-opening and connection cost obtained by applying the standard SimpleUniformFL post-processing (line 4–6 in SIMPLEUNIFORMFL) using the probabilities $\{p_x\}_{x \in X}$. In particular, the mpnn_{θ^} realizes an $\mathcal{O}(\log(n))$ -approximation algorithm for the UniFL on all instances of order n .*

Proof. We verify the result in three steps: (i) show that the MPNN can approximate the radii, (ii) show that the MPNN can compute the opening probabilities, and (iii) show that we can recover the approximation ratio up to a constant, which we outline below. Let $(\mathcal{X}, d)$ be a finite metric space with $n := |\mathcal{X}|$.

The MPNN approximates the radii Recall that the radius $r_x \in (0,1]$, for $x \in \mathcal{X}$, defined as the unique solution of

$$\sum_{y \in X \cap B(x, r_x)} (r_x - d(x, y)) = 1. \tag{7}$$

By construction, the function

$$\varphi_x(r) := \sum_{y \in X \cap B(x,r)} (r - d(x,y))_+, \quad (t)_+ := \max\{0, t\},$$

is continuous and strictly increasing in $r \in (0,1]$, and r_x is the unique point with $\varphi_x(r_x) = 1$.

Now, for each bin boundary a_i , we define the

$$t_x^{(i)} := \min \left\{ 1, \sum_{y \in N(x)} (a_i - d(x,y))_+ \right\} = 1 - \left(1 - \sum_{y \in N(x)} (a_i - d(x,y))_+ \right)_+.$$

By construction, $t_x^{(i)}$ depends only on x 's neighbors in G_S , and both the inner map $(a_i, d(x,y)) \mapsto (a_i - d(x,y))_+$ and the outer map $z \mapsto \min\{1, z\}$ can be implemented by constant-depth, constant-width FNNs with ReLU activation, which can be verified readily, following the construction in Section 3.2. Hence, an MPNN with sum aggregation and two FNN layers can compute all values $\{t_x^{(i)}\}_{i \in [k]}$ in parallel.

Consider Equation (4), because $\varphi_x(\cdot)$ is strictly increasing and crosses the threshold value 1 exactly once, the sequence $\{t_x^{(i)}\}_{i=0}^k$ changes from values strictly below 1 to values equal to 1 at (or very close to) the true radius r_x . With the given discretization, this implies that there exists an index $j \in [k]$ such that

$$a_{j-1} \leq r_x \leq a_j \quad \text{and} \quad \hat{r}_x \in [a_{j-1}, a_j].$$

Now, let,

$$\Delta := \max_{i \in [k]} (a_i - a_{i-1}).$$

Hence,

$$|\hat{r}_x - r_x| \leq \Delta.$$

Consequently, overall,

$$\left| \sum_{x \in X} \hat{r}_x - \sum_{x \in X} r_x \right| \leq n\Delta.$$

Hence, by choosing a discretization such that Δ is sufficiently small, we can make the absolute difference in Appendix C.2 arbitrarily small.

Computing opening probabilities from radii The SIMPLEUNIFORMFL algorithm in Section 3.1 opens a facility at x in line 2 with probability

$$p_x^* := \min\{1, c \log(n) r_x\}$$

for a suitable constant $c > 0$. Our MPNN uses the approximate quantities $\hat{r}_x$ and computes

$$p_x := \min\{1, c \log(n) \hat{r}_x\} = 1 - (1 - c \log(n) \hat{r}_x)_+.$$

Again, the mapping $(\log(n), \hat{r}_x) \mapsto p_x$ is realized by a constant-depth, constant-width FNN with ReLU activation, applied point-wise to the vertex features produced by the above-constructed MPNN, following Section 3.2. Hence, overall, we have constructed the desired MPNN mpnn_{θ^*} , with parameter assignment θ^* , such that

$$|p_x - p_x^*| \leq c \log(n) |\hat{r}_x - r_x| \leq c \log(n) \Delta, \quad \text{for all } x \in \mathcal{X}. \quad (8)$$

Again, by choosing a discretization such that Δ is sufficiently small, we can arbitrarily closely approximate p_x^* , for $x \in \mathcal{X}$.

Approximation guarantee Recall from Section 3.1 that if we denote by F the random set of facilities opened in line 2 of SIMPLEUNIFORMFL, and by A_f and B_f the indicator variables for the events “ $f \in F$ ” and “no facility within distance < 1 of f is open after line 2,” then the expected opening cost with probabilities $\{p_x\}$ can be written as

$$\mathbb{E}[\text{open}(X, p)] = \sum_{f \in X} p_f + \sum_{f \in X} \prod_{x \in X: d(x,f) < 1} (1 - p_x),$$

and the expected connection cost admits the expression

$$\mathbb{E}[\text{conn}(X, p)] = \sum_{x \in X} \sum_{f \in X} d(x, f) p_f \prod_{z \in X: d(x, z) < d(x, f)} (1 - p_z),$$

so that the total expected cost is

$$\mathbb{E}[\text{cost}(X, p)] = \mathbb{E}[\text{open}(X, p)] + \mathbb{E}[\text{conn}(X, p)].$$

Now let

$$\Phi(p) := \mathbb{E}[\text{cost}(X, p)].$$

By Equation (8), for every $\varepsilon > 0$, there exists Δ_ε such

$$|\Phi(p) - \Phi(p^*)| \leq \varepsilon. \quad (9)$$

On the other hand, Lemma 1 shows that

$$\sum_{x \in X} r_x \in \Theta(\text{Opt}_S),$$

and Proposition 2 applied to SIMPLEUNIFORMFL with the ideal probabilities $\{p_x^*\}$ yields

$$\mathbb{E}[\text{cost}(X, p^*)] \leq C \log(n) \cdot \text{Opt}_S,$$

for some $C > 0$. Hence, using Equation (9) and the fact that we can absorb the additive term ε into the multiplicative constant in front of $\log(n) \cdot \text{Opt}_S$. That is, there exists a universal constant $C_\varepsilon > 0$ such that

$$\mathbb{E}[\text{cost}(X, p)] \leq C_\varepsilon \log(n \cdot \text{Opt}_S).$$

This shows that M_θ realizes an $O(\log(n))$ -approximation algorithm for UniFL on all instances of order n , completing the proof. $\square$

Lemma 9 (Proposition 4 in the main paper). *Let mpnn be an arbitrary (deterministic) MPNN that*

1. *receives as input an edge-weighted graph G_S that encodes an instance $\mathcal{S} = (X, d)$ of the uniform facility location problem, and*
2. *computes opening probabilities p_v for every vertex $v \in V$.*

Then there is n_0 such that for every $n \geq n_0$ there exists an instance finite metric space $S := (\mathcal{X}, d)$, with $|\mathcal{X}| = n$, such that

$$\mathbb{E} \left[|F| + \sum_{p \in X} \min\{1, \text{dist}(p, F)\} \right] \geq \frac{\ln(n)}{2} \cdot \text{Opt}_S.$$

Proof. We consider an n -point metric space $\mathcal{X}$ in which every pair of points from $\mathcal{X}$ has pairwise distance $\varepsilon > 0$ for an arbitrary small $\varepsilon > 0$. An optimal solution (for sufficiently small $\varepsilon > 0$) opens 1 facility and has cost $1 + \varepsilon n$. Since the graph G_S is symmetric and the MPNN mpnn is deterministic, it holds that there is a value p such that $p_v = p$, for $v \in V(G_S)$. Then we get

$$\mathbb{E} \left[|F| + \sum_{p \in X} \min\{1, \text{dist}(p, F)\} \right] \geq pn + (1 - p)^n n \geq pn + \left(\frac{1}{e}\right)^{pn} n \geq \frac{\ln n}{2} \cdot \text{Opt}_S.$$

$\square$

C.3. Analysis of the recursive algorithm

Here, we prove the $\mathcal{O}(1)$ -approximation factor of UNIFORMFLRECURSIONSTART.

Lemma 10. *Let $(\mathcal{X}, d)$ be a UniFL instance. If we run the algorithm UNIFORMFLRECURSIONSTART on $(\mathcal{X}, d)$ then for each recursive call of RECURSIVEUNIFORMFL with parameters $\mathcal{X}'$, F and d we have for every $x \in \mathcal{X}'$ that $B_{\mathcal{X}'}(x, r_x) \subseteq \mathcal{X}'$ or $d(x, F) \leq 13r_x$, where the radius r_x is defined with respect to set $\mathcal{X}$.*

Proof. For $x \in \mathcal{X}$ let y be the first point from $B_{\mathcal{X}}(x, r_x)$ that is assigned to a facility. Consider the corresponding call of RECURSIVEUNIFORM. Let $\mathcal{X}' \subseteq \mathcal{X}$ be the set of points passed to this call. Since y is the first point from $B_{\mathcal{X}}(x, r_x)$ that is assigned to a facility we know that $B_{\mathcal{X}}(x, r_x) \subseteq \mathcal{X}'$. Hence, the value of r_x remains the same for $\mathcal{X}$ and $\mathcal{X}'$. Now consider the current r_y for $y \in B_{\mathcal{X}'}(x, r_x) = B_{\mathcal{X}}(x, r_x)$ that are defined wrt. $\mathcal{X}'$. We claim that $r_y \leq 2r_x$ as otherwise

$$1 = \sum_{p \in B_{\mathcal{X}'}(x, r_x)} r_y - d(p, y) > \sum_{p \in B_{\mathcal{X}'}(x, r_x)} 2r_x - (d(p, x) + d(x, y)) \geq \sum_{p \in B_{\mathcal{X}'}(x, r_x)} r_x - d(p, x) = 1.$$

At the point in time y gets assigned, there is a facility within distance $6r_y$ from y and so there is a facility within distance $13r_x$ from x as $d(x, y) \leq r_x$. During further recursive calls, x will therefore be assigned to a facility within distance at most $13r_x$. $\square$

Corollary 11. *Let $(\mathcal{X}, d)$ be a UniFL instance. If we run the algorithm UNIFORMFLRECURSIONSTART on $(\mathcal{X}, d)$ then*

$$\sum_{x \in \mathcal{X}} d(x, F) \in O(\text{OPT}),$$

where OPT denotes the objective value of an optimal solution.

Proof. Follows from Lemma 10 and Lemma 1. $\square$

We now continue to analyze the opening cost, i.e., $|F|$. We will use Lemma 6

Lemma 12. *Consider an arbitrary call to RECURSIVEUNIFORMFL with $c \geq 6$ with parameters $(\mathcal{X}, F', d)$. Let F and R denote the returned sets of points. For every $x \in \mathcal{X}$ we have*

$$\Pr(x \in R) \leq 1/10.$$

Proof. Let $x \in \mathcal{X}$ be arbitrary. If $d(x, F') \leq 6r_x$ then x will be assigned to an open facility with probability 1. Thus, we can assume that $d(x, F') > 6r_x$. By Lemma 6 there is a set T of k points within distance at most $2r_x$ from x and $k \geq 1/(2r_y)$ for all $y \in T$. We observe that for all $y \in T$ we have $d(y, F') > 4r_x$ by the triangle inequality. By a similar argument (the ball around y cannot contain the ball around x) as in the previous proof, each $y \in T$ has $r_y \leq 4r_x$. Hence, $r_y \leq d(y, F')$ and so we select each $y \in T$ with probability at least $\min\{1, c \cdot d(y, F'), c \cdot r_y\} = \min\{1, cr_y\}$.

The probability that none of these points has been opened is at most $(1 - \min(1, \frac{c}{2k}))^k$. If $k < \frac{c}{2}$ this probability is 0, so we assume in the following that $k \geq \frac{c}{2}$. In this case, we get $(1 - \min(1, \frac{c}{2k}))^k = (1 - \frac{c}{2k})^k \leq e^{-k \frac{c}{2k}} = e^{-\frac{c}{2}} = \frac{1}{e^3} \leq 1/10$, for $c \geq 6$. $\square$

Now, the following results show that UNIFORMFLSTART indeed yields a constant-factor approximation for UniFL.

Lemma 13. *Consider any call to UNIFORMFLSTART with parameters $(\mathcal{X}, F, d)$. Then*

$$\mathbb{E}[|F|] \in \mathcal{O}(\text{OPT}),$$

where OPT denotes the cost of an optimal solution.

Proof. By Lemma 12 we get that the probability that a facility is opened at x in the i -th recursive call is at most $(1 - \frac{1}{10})^i \cdot c \cdot d(x, F)$. By Lemma 10 this probability is at most $(1 - \frac{1}{10})^i \cdot 13 \cdot c \cdot r_x$. Let $Y_x^{(i)}$ be the indicator random variable that we

open a facility at x in the i -th recursion, and let $Y_x = \sum_{i=1}^{\infty} Y_x^{(i)}$ be the indicator random variable that we open a facility at x . Hence, $\mathbb{E}[Y_x] \leq \sum_{i=1}^{\infty} (1 - \frac{1}{10})^i \cdot c \cdot d(x, F)$. We get

$$\mathbb{E}[|F|] = \sum_{x \in \mathcal{X}} \mathbb{E}[Y_x] \in \mathcal{O}\left(\sum_{x \in \mathcal{X}} r_x\right) = \mathcal{O}(\text{OPT}).$$

□

C.4. Size generalization guarantees

Here, we prove the size generalization guarantees for the MPNN architecture.

Proposition 14 (Proposition 5 in the main paper). *For every $n \in \mathbb{N}$, there exists an MPNN architecture $\text{mpnn}_{\theta}^{(n)}: \mathcal{V}(\mathcal{G}_n^C) \rightarrow [0,1]$, with parameters θ , such that for $\varepsilon \in (0,1)$, there is $\varepsilon' > 0$, and a finite training datasets $T_{n,\varepsilon} \subset \mathcal{V}(\mathcal{G}_n^C) \times \mathbb{R}$ consisting of pairs $((G, v), p_v)$, where $G \in \mathcal{G}_n^C$ is a finite, edge-weighted graph encoding an UFL instance, $v \in V(G)$, and p_v is the corresponding opening probability according to Proposition 3, and a differentiable regularization term r_n such that, such that*

$$\frac{1}{|T_{n,\varepsilon}|} \sum_{((G,v),p_v) \in T_n} |\text{mpnn}_{\theta}^{(n)}(G, v) - p_v| + r_n(\theta) \leq \varepsilon'$$

implies

$$\|p_{G_S} - h_{\theta}(G_S)\| \leq \varepsilon,$$

for all $G_S \in \mathcal{G}_n^C$ encoding UniFL instances.

Proof sketch. We essentially use the MPNN architecture from Section 3.2; however, we use order-normalized sum aggregation instead of sum aggregation. That is, given a suitable discretization, we compute

$$t_x^{(i)} := \min \left\{ \frac{1}{n}, \frac{1}{n} \sum_{y \in N(x)} \text{ReLU}(a_i - d(x, y)) \right\} = \frac{1}{n} - \text{ReLU} \left(\frac{1}{n} - \sum_{y \in N(x)} \text{ReLU}(a_i - d(x, y)) \right),$$

for $i \in [k]$. Following Section 3.2, we can further parametrize with a two-layer FNN using ReLU activation functions as follows,

$$t_x^{(i)} \equiv \text{FNN}_{2,3} \left(\frac{1}{n} \sum_{y \in N(x)} \text{FNN}_{1,3}(a_i, d(x, y)) \right).$$

The remaining parts of the architecture stay the same. Let $\text{mpnn}_{\theta}^{(n)}: \mathcal{V}(\mathcal{G}_n) \rightarrow \mathbb{R}$ be the resulting MPNN architecture and $\mathcal{F}_{\Theta} := \{\text{mpnn}_{\theta}^{(n)} \mid \theta \in \Theta\}$, for some appropriately chosen parameter set Θ .

The rest of the proof follows the technique developed in Wittig et al. (2026). That is, implied by (Rauchwerger et al., 2025), assuming the employed FNNs are Lipschitz continuous, there exists a pseudo-metric d on $\mathcal{V}(\mathcal{G}_n)$ such that for each $\text{mpnn}_{\theta}^{(n)} \in \mathcal{F}_{\Theta}$, there is B_{θ} such that $\text{mpnn}_{\theta}^{(n)}$ is Lipschitz continuous with Lipschitz constant M_{θ} regarding the pseudo-metric d and the metric space $(\mathbb{R}, |\cdot|)$. In addition, for all $\varepsilon > 0$, there exists a finite covering number $K_{\varepsilon} := \mathcal{N}(\mathcal{X}, d, \varepsilon)$.

Observe that, analogous to Proposition 3, we can find parameters θ^* such that the expected costs are an $\mathcal{O}(\log(n))$ -factor approximation. Let M^* be the Lipschitz constant for $\text{mpnn}_{\theta^*}^{(n)}$. Now, set the training set equal to an r -cover of the space $(\mathcal{X}, d)$ and set

$$r_n(\theta) := \eta \text{ReLU}(M_{\theta} - M^*),$$

for $\eta > 0$. Assume there is a $\theta \in \Theta$, such for $\text{mpnn}_{\theta}^{(n)}$, the above regularized loss is smaller than ε' . Hence, by exploiting the Lipschitzness and the triangle inequality, we get

$$\|\text{mpnn}_{\theta}^{(n)} - \text{mpnn}_{\theta^*}^{(n)}\|_{\infty} \leq (M_{\theta} + M^*)r + K_r \varepsilon', \quad (10)$$

and that

$$M_{\theta} \leq \frac{1}{\eta} \varepsilon' + M^*. \quad (11)$$

Now, choose $r = \frac{\varepsilon}{6(1+B_{f^*})}$ and $\varepsilon' < \min \left\{ \frac{\varepsilon}{3K_r}, \frac{\varepsilon\eta}{6(1+B_{f^*})} \right\}$. By Equation (11), we get

$$M_{\theta} \leq M^* + \frac{\varepsilon'}{\eta} = M^* + \frac{\varepsilon}{6(1+M^*)}.$$

Hence, by using Equation (10), and the assumption $\varepsilon < 1$, and straightforward calculations, we get

$$\|\text{mpnn}_{\theta}^{(n)} - \text{mpnn}_{\theta^*}^{(n)}\|_{\infty} \leq \varepsilon.$$

□

D. Details on experiments and additional results

Here, we give additional details on the experiments and discuss additional results.

Table 8. Statistics of datasets.

Name	#node	#degree	#dimension
Geo-1000-2	1000	6.8	2
Geo-2000-2	2000	7.3	2
Geo-3000-2	3000	6.7	2
Geo-5000-2	5000	6.8	2
Geo-10000-2	10000	7.0	2
Geo-1000-5	1000	7.2	5
Geo-1000-10	1000	10.8	10
Geo-2000-10	2000	10.4	10
Geo-3000-10	3000	10.2	10
Geo-5000-10	5000	9.6	10
Geo-10000-10	10000	10.4	10
Geo-1000-10-dense	1000	34.9	10
Geo-1000-10-sparse	1000	2.1	10
Paris	114127	4.2	–
Shanghai	183917	3.8	–
London	568795	3.7	–
LA	240587	3.8	–

Baselines We use the optimal solution obtained via an ILP solver as our gold-standard baseline. Specifically, given a UniFL instance S , we define binary decision variables y_i for $i \in [n]$ to indicate whether a facility is opened at vertex i . Additionally, we define binary assignment variables e_{ij} for $(i, j) \in E(G_S)$ to indicate whether the client at vertex i is assigned to the facility at vertex j . The parameter w_{ij} represents the edge distance; notably, the edge set $E(G_S)$ includes self-loops with $w_{ii} = 0$ to allow for self-service. The UniFL problem is formulated as follows,

$$\begin{aligned} \arg \min_{\mathbf{y}, \mathbf{e}} \quad & \sum_{i \in [n]} y_i + \sum_{(i, j) \in E} w_{ij} e_{ij} \\ \text{s.t.} \quad & e_{ij} \leq y_j \\ & \sum_{j: (i, j) \in E(G_S)} e_{ij} = 1 \\ & y_i, e_{ij} \in \{0, 1\} \end{aligned}$$

The first constraint ensures that vertex i can be assigned to vertex j only if a facility is open at j . The second constraint guarantees that every location is assigned to exactly one facility, potentially itself.

Experimental protocol We employ Google’s OR-Tools (Perron & Furnon) as the exact ILP solver, imposing a 24-hour time limit per instance. For the $\mathcal{O}(1)$ -UFL (Gehweiler et al., 2014) baseline, we repeat the stochastic process with 1000 seeds

and average the result. For the baselines SIMPLEUNIFORMFL and UNIFORMFLRECURSIONSTART, we precompute vertex radii and estimate the expected cost by averaging results over 1000 sampling seeds per instance. To ensure termination in UNIFORMFLRECURSIONSTART, we limit the while-loop to 100 iterations, and if any vertex has no facilities in the neighborhood (including itself) after the loop limit, we force a facility to open at its location. Both baseline methods rely on a single scalar hyperparameter to scale the facility-opening probability. We sweep this parameter using a grid search over 100 candidate values, logarithmically spaced over the interval $[0.001, 10]$. We report the final performance as the average over the entire test dataset.

For our MPNN approach, we leverage an MPNN with 32 hidden dimensions for all datasets, six layers for geometric graphs, and ten layers for city map datasets. We train it with the unsupervised loss in Equation (5) for at most 1000 epochs and 100 patience for early stopping. The entire training process is repeated five times using independent random seeds. The trained MPNN then guides the sampling algorithm in SIMPLEUNIFORMFL. Specifically, we drop the c constant, predict the facility-opening probability directly using the MPNN, and repeat the sampling 1000 times with different seeds. We aggregate these results by first averaging over the 1000 samples per graph, then averaging over the dataset. Finally, we report the mean and standard deviation of these dataset-level scores across five training seeds. To enable fair comparison across different problem scales, we also report the mean optimality ratio, defined as the average ratio of a method’s cost to the exact solution provided by the solver. For detailed hyperparameters, please see to Table 9.

Table 9. Hyperparameters of baselines and MPNN approach.

Name	SimpleUFL	RecurUFL	MPNN layer	MPNN dim.
Geo-1000-2	0.560	0.037	6	32
Geo-1000-5	0.423	0.166	6	32
Geo-1000-10	0.351	0.242	6	32
Geo-1000-10-dense	0.319	0.291	6	32
Geo-1000-10-sparse	0.423	0.031	6	32
Paris	1.417	0.095	10	32
Shanghai	1.417	0.095	10	32
London	1.292	0.086	10	32
LA	1.292	0.086	10	32

On the geometric graph datasets, we further investigate a variant of our unsupervised approach tailored to the k -Means objective. By squaring the distance term $\text{dist}(x, f)$ in the connection-cost component of the loss function Equation (5), we effectively shift the optimization goal to minimizing the sum of squared errors, treating open facilities as cluster centroids. For this comparative study, we train the MPNN using this modified squared-distance loss while keeping all other training protocols identical. During inference, we use the MPNN to guide SIMPLEUNIFORMFL and evaluate the resulting connection cost as the sum of squared distances. To benchmark this approach, we employ standard clustering algorithms k -Means and k -Medoids (Pedregosa et al., 2011) as baselines.¹ For each trained MPNN, we set the number of clusters k for these baselines equal to the expected number of facilities predicted by the MPNN. We utilize the k -Means++ and k -Medoids++ initialization, respectively, running 1000 independent initializations per instance and reporting the average result. Furthermore, we use the GNN-predicted facility locations as initializations for k -Means and k -Medoids. Finally, we report the mean and standard deviation of the performance across the five runs for both the GNN and the clustering baseline; see Algorithm 1 for a reference algorithm.

For a fair comparison of computational efficiency, we report the wall-clock time required to generate a one-shot solution rather than to repeat the sampling. For SIMPLEUNIFORMFL and $\mathcal{O}(1)$ -UFL (Gehweiler et al., 2014), we measure the time for radii pre-computation plus one sampling process. Similarly, for our GNN-guided approach, we record the neural network inference time and the time required for a single sampling step. Finally, UNIFORMFLRECURSIONSTART is timed on a single execution, including the necessary while-loops up to a limit of 100.

All stochastic baselines and GNN training and inference are executed on a single NVIDIA L40S GPU. The exact ILP solver, as well as k -Means and k -Medoids, runs on an Intel® Xeon® Silver 4510 CPU, as it lacks GPU support. We acknowledge that using different hardware platforms makes a direct speed comparison inexact; however, we report raw runtimes for completeness. We note that the ability to leverage GPU acceleration for massive parallelism is a distinct advantage of our MPNN approach.

¹The k -Medoids metric is the Euclidean distance; in our case, we adjusted it to the squared Euclidean distance.

Algorithm 1 Algorithm for clustering comparison. We compare our MPNN prediction against k -Means and k -Medoids with k -Means++ and k -Medoids++ initializations, and also with MPNN-predicted facilities as initialization.

Require: Graph $G = (V, E)$, vertex location matrix $\mathbf{X}$, trained MPNN model Φ , repeat count R

Ensure: Sum squared costs for MPNN, k -Means++ and k -Medoids++, and MPNN-initialized k -Means and k -Medoids

- 1: Initialize lists $L_{\text{MPNN}}, L_{\text{mean}}, L_{\text{med}}, L_{\text{mean}}^{\text{init}}, L_{\text{med}}^{\text{init}} \leftarrow \emptyset$
- 2: $\mathbf{p} \leftarrow \Phi(G)$ $\triangleright$ Predicted vertex opening probability
- 3: **for** $r \leftarrow 1$ **to** R **do**
- 4: **Step 1: MPNN Sampling**
- 5: Sample $\mathbf{y} \sim \text{Bernoulli}(\min(\mathbf{p}, 1))$
- 6: **if** $\exists v$ s.t. $\{N(v) \cup v\} \cap \{u \mid y_u = 1\} = \emptyset$ **then**
- 7: $y_v \leftarrow 1$ $\triangleright$ If v has no facility in neighborhood, force it open
- 8: $k \leftarrow \sum y_v$
- 9: $\mathbf{X}_{\text{init}} \leftarrow \{\mathbf{x}_v \mid y_v = 1\}$ $\triangleright$ MPNN-predicted centroids
- 10: Append MPNN squared distance cost to L_{MPNN}
- 11: **Step 2: k -Means++ / k -Medoids++**
- 12: Run k -Means with k -Means++ initialization; append inertia to L_{mean}
- 13: Run k -Medoids with k -Medoids++ initialization; append inertia to L_{med}
- 14: **Step 3: MPNN-guided k -Means / k -Medoids**
- 15: Run k -Means with $\mathbf{X}_{\text{init}}$ initialization; append inertia to $L_{\text{mean}}^{\text{init}}$
- 16: Run k -Medoids with $\mathbf{X}_{\text{init}}$ initialization; append inertia to $L_{\text{med}}^{\text{init}}$
- 17: **return** Means of all lists

Table 10. Results on size generalization, shown in mean $\pm$ standard deviations over five runs. MPNN architectures are trained on Geo-1000-2 and tested on larger instances of 2D Euclidean space.

Candidate	Cost	Geo-1000-2	Geo-2000-2	Geo-3000-2	Geo-5000-2	Geo-10000-2
Solver	Open	366.302	745.330	1201.360	1989.970	3980.100
	Con.	279.827	548.501	813.603	1346.195	2677.519
	Total	646.129	1293.831	2014.963	3336.165	6657.619
	Time	0.263	0.831	1.329	3.630	13.971
MPNN	Open	380.816 $\pm$ 0.761	774.047 $\pm$ 1.065	1265.818 $\pm$ 5.796	2089.730 $\pm$ 6.018	4180.888 $\pm$ 15.306
	Con.	271.317 $\pm$ 0.735	532.306 $\pm$ 1.060	772.892 $\pm$ 4.517	1284.182 $\pm$ 4.979	2553.858 $\pm$ 12.836
	Total	652.133 $\pm$ 0.025	1306.354 $\pm$ 0.012	2038.710 $\pm$ 1.282	3373.912 $\pm$ 1.065	6734.747 $\pm$ 2.488
	Time	0.004 $\pm$ 0.000	0.005 $\pm$ 0.000	0.006 $\pm$ 0.000	0.006 $\pm$ 0.000	0.008 $\pm$ 0.000
	Ratio	1.009 $\pm$ 0.000	1.009 $\pm$ 0.000	1.011 $\pm$ 0.000	1.011 $\pm$ 0.000	1.012 $\pm$ 0.000